



ESCUELA DE INGENIERÍA DE FUENLABRADA

GRADO EN INGENIERÍA DE
ROBÓTICA SOFTWARE

TRABAJO FIN DE GRADO

IA EXPLICATIVA APLICADA A LA DETECCIÓN
DE EMOCIONES A TRAVÉS DE SEÑALES EEG

Autor: Liam Saborido Sueiro

Tutor: Álvaro García López

Co-Tutor: Julio Salvador Lora Millán

Curso académico 2025-2026



Este trabajo se distribuye bajo los términos de la licencia internacional CC BY-SA International License (Creative Commons Attribution-ShareAlike 4.0). Usted es libre de *(a) compartir*: copiar y redistribuir el material en cualquier medio o formato para cualquier propósito, incluso comercialmente; y *(b) adaptar*: remezclar, transformar y construir a partir del material para cualquier propósito, incluso comercialmente. La licenciante no puede revocar estas libertades en tanto usted siga los términos de la licencia:

- *Atribución.* Usted debe dar crédito de manera adecuada, brindar un enlace a la licencia, e indicar si se han realizado cambios. Puede hacerlo en cualquier forma razonable, pero no de forma tal que sugiera que usted o su uso tienen el apoyo de la licenciante.
- *Compartir igual.* Si remezcla, transforma o crea a partir del material, debe distribuir su contribución bajo la misma licencia del original.
No hay restricciones adicionales — No puede aplicar términos legales ni medidas tecnológicas que restrinjan legalmente a otras a hacer cualquier uso permitido por la licencia.

Agradecimientos

Quiero dar las gracias a todas las personas que han estado conmigo durante estos años. A mis padres, Susana y José, por su ayuda económica y emocional, y por confiar en mí incluso cuando yo no lo tenía tan claro. A mi hermana, Xiana, que siempre me ha dado los mejores consejos, especialmente en una etapa en la que tuve que aprender muchas cosas. A mi pareja, Ioana, por convertir muchos días malos en momentos buenos, hacerme reír cuando más lo necesitaba y apoyarme incondicionalmente. Y a mis amigos que siempre me recordaron que no todo era estudiar o preocuparse.

También quiero agradecer a las personas involucradas en la formación del conjunto de datos en la Universidad Complutense. Sin ese trabajo previo, literalmente me habría sido imposible realizar este proyecto. La creación de un conjunto de datos así requiere mucho tiempo, y gran parte de ese esfuerzo no siempre se ve cuando uno trabaja con los datos ya preparados. Y, obviamente, a mi tutor, Álvaro, por su guía, su tiempo y su ayuda durante el desarrollo de este trabajo.

Este trabajo lleva mi nombre, pero no habría llegado a hacerlo sin todos vosotros...

A mi familia, mi pareja y mis amigos

Madrid, 1 de Mayo de 2026

Liam

Resumen

Aun no

Acrónimos

GSR respuesta galvánica de la piel

EMG electromiografía

EOG electrooculografía

FC frecuencia cardíaca

FR frecuencia respiratoria

EEG electroencefalografía

MEG magnetoencefalografía

EDF *European Data Format*

FIF *Functional Imaging File*

JSON *JavaScript Object Notation*

PSD *Power Spectral Density*

FFT *Fast Fourier Transform*

PCA *Principal Component Analysis*

LDA *Linear Discriminant Analysis*

GPU *Graphics processing unit*

fMRI resonancia magnética funcional

PET tomografía por emisión de positrones

ICA *Independent Component Analysis*

SHAP *SHapley Additive exPlanations*

Índice general

1. Introducción	1
1.1. Contexto y motivación	1
1.2. Electroencefalografía como fuente de información afectiva	2
1.3. Reconocimiento automático de emociones	3
1.4. IA de caja blanca e interpretabilidad en señales biomédicas	3
2. Objetivos y planificación	5
2.1. Descripción y objetivo	5
2.2. Requisitos	6
2.3. Metodología de trabajo	7
2.4. Planificación y seguimiento	8
3. Fundamentos teóricos	9
3.1. Señales EEG y posicionamiento de electrodos	9
3.2. Bandas espectrales	11
3.3. Emociones y actividad cerebral: alegría, tristeza y estado neutro	12
3.4. Aprendizaje automático aplicado a EEG	14
3.4.1. Modelos interpretables frente a modelos de caja negra	15
3.4.2. Clasificadores	16
3.4.3. Estrategia de evaluación y validación cruzada	18

3.4.4. Métricas	20
3.5. Técnicas de explicación	23
4. Materiales, datos y entorno de trabajo	25
4.1. Protocolo experimental y estímulos auditivos	25
4.2. Descripción general de los datos	26
4.3. Organización del repositorio	27
4.4. Entorno de trabajo	30
4.4.1. Entorno físico y software	30
4.4.2. Librerías y dependencias principales	31
5. Desarrollo del sistema	33
5.1. Flujo general del sistema	33
5.2. Preprocesado de registros EEG	34
5.2.1. Carga de archivos y extracción de eventos	35
5.2.2. Selección de canales, renombrado y montaje EEG	35
5.2.3. Filtrado y remuestreo	36
5.2.4. Detección de canales problemáticos y corrección de artefactos	37
5.2.5. Segmentación en épocas y rechazo de segmentos anómalos	39
5.2.6. Almacenamiento de épocas y estadísticas de control	39
5.2.7. Resumen	40
5.2.8. Control de sujetos válidos	40
5.3. Extracción de características	41
5.3.1. Carga de épocas y sujetos válidos	42
5.3.2. Cálculo de potencia espectral por bandas	42
5.3.3. Normalización respecto a la condición neutra	44

5.3.4.	Construcción del conjunto de características	46
5.3.5.	Guardado y check	49
5.3.6.	Resumen	50
5.4.	Entrenamiento y evaluación de modelos	50
5.4.1.	Entrada al entrenamiento	51
5.4.2.	Construcción de características dentro de cada partición	51
5.4.3.	Clasificadores utilizados	52
5.4.4.	Entrenamiento <i>within-subject</i>	53
5.4.5.	Entrenamiento <i>all-subjects</i>	54
5.4.6.	Métricas, figuras y modelos guardados	55
5.4.7.	Resumen	56
5.5.	Análisis de separabilidad	57
5.5.1.	Entrada y construcción del espacio de análisis	57
5.5.2.	Proyecciones globales y por sujeto	58
5.5.3.	Métricas de separabilidad	58
5.5.4.	Resumen	59
5.6.	Explicabilidad de los modelos	59
5.6.1.	Cálculo de explicaciones out-of-fold	60
5.6.2.	Agregación de importancias	61
5.6.3.	Representaciones generadas	62
5.6.4.	Guardado de resultados	62
5.6.5.	Resumen	64

Índice de figuras

3.1. Principales lóbulos de la corteza cerebral. Fuente: [7].	10
3.2. Sistema internacional 10-20 para la colocación de electrodos EEG. Fuente: [10].	11
3.3. Representación simplificada de emociones según valencia y arousal. Adaptado de [11].	13
3.4. Comparación entre regresión logística binaria y multiclase.	17
3.5. Ejemplo de estructura de árbol de decisión. Fuente: [17].	17
3.6. Esquema de validación cruzada <i>k-fold</i> . Fuente: [19].	19
3.7. Ejemplo de matriz de confusión binaria. Fuente: [23].	21
3.8. Ejemplo de topomaps por banda espectral para la clase alegría.	23
3.9. Ejemplo de importancia de características mediante valores SHAP.	24
4.1. Entorno y material utilizados durante el registro de las señales EEG.	26
5.1. Flujo general del sistema desarrollado.	33
5.2. Resumen del flujo de preprocesado aplicado a cada registro electroencefalografía (EEG).	40
5.3. Ejemplo de señal en dominio temporal y frecuencial	43
5.4. Resumen del flujo de extracción de características aplicado a las épocas preprocesadas.	50
5.5. Resumen del flujo de entrenamiento y evaluación de modelos.	56

5.6. Resumen del flujo de análisis de separabilidad.	59
5.7. Flujo seguido por el script de explicabilidad para calcular importancias out-of-fold y generar las salidas de interpretación.	64

Índice de tablas

3.1. Bandas espectrales principales de la señal EEG.	12
5.1. Ejemplo de transformación logarítmica en base 10.	45
5.2. Ejemplo simplificado de la estructura del conjunto de características. .	48

Listado de códigos

Listado de ecuaciones

3.1. Probabilidad estimada por una regresión logística binaria.	16
3.2. Promedio de una métrica en validación cruzada <i>k-fold</i>	19
3.3. Exactitud en clasificación multiclase a partir de la matriz de confusión.	21
3.4. Precisión, recall y F1-score para una clase <i>c</i>	22
3.5. F1 macro como promedio no ponderado del F1-score por clase.	22
5.1. Cálculo del z-score para una métrica de canal.	38
5.2. Transformada discreta de Fourier calculada de forma eficiente mediante FFT.	42
5.3. Estimación de la densidad espectral de potencia mediante el método de Welch.	43
5.4. Potencia media de una banda de frecuencia para una época y un canal.	44
5.5. Transformación logarítmica de la potencia espectral.	45
5.6. Referencia neutra de un sujeto para cada canal y banda.	45
5.7. Diferencia logarítmica respecto a la condición neutra del mismo sujeto.	45
5.8. Normalización z-score por sujeto usando la condición neutra como referencia.	46
5.9. Relación Theta/Alpha calculada como diferencia en escala logarítmica.	47
5.10. Asimetría Alpha entre un par de canales homólogos.	48

5.11. Contribución <i>SHapley Additive exPlanations</i> (SHAP) lineal utilizada para la regresión logística en cada fold.	60
--	----

Capítulo 1

Introducción

Este capítulo presenta la idea general, la motivación detrás del tema y los conceptos principales en los que se basa. Se habla de la electroencefalografía como fuente de información para detectar emociones, del reconocimiento automático de emociones y de la importancia de usar modelos que se puedan interpretar. Por último, se explica cómo está organizada la memoria.

1.1. Contexto y motivación

La electroencefalografía destaca por su capacidad para extraer datos de un órgano tan delicado como el cerebro sin necesidad de realizar una intervención invasiva. Esto la convierte en una herramienta muy valiosa en la neurología, ya sea para diagnosticar tumores y lesiones cerebrales, evaluar el deterioro cognitivo o incluso detectar emociones [1].

Cuando era pequeño me hicieron una prueba del sueño en la que me colocaron electrodos con una pasta adhesiva bastante difícil de quitar y me monitorizaron durante toda la noche. Me llamó mucho la atención: ¿cómo podía un médico mirar todos esos “garabatos” llamados señales y entender qué pasaba en mi cabeza?

Con el tiempo, esa curiosidad se juntó con mi interés por la inteligencia artificial y su uso en la medicina. Al ver cómo otras personas creaban modelos capaces de detectar tumores, me pregunté si también sería posible hacer algo parecido con electroencefalogramas. La idea no es sustituir a los médicos, sino crear una herramienta que pueda ayudarles a interpretar mejor grandes cantidades de datos y aportar información útil de forma automática y fiable.

Este trabajo se centra en el reconocimiento de emociones mediante señales de EEG,

dando importancia no solo a los resultados obtenidos, sino también a la interpretación de los modelos. En el ámbito biomédico, no basta con que un modelo clasifique bien: también es necesario entender qué información está utilizando y si sus decisiones pueden explicarse de forma clara.

1.2. Electroencefalografía como fuente de información afectiva

Desde siempre, las emociones se han intentado interpretar a partir de lo que una persona dice, sus gestos o sus expresiones faciales. Sin embargo, en un contexto médico esto no siempre es suficiente, ya que una persona puede no expresar exactamente lo que siente. En consecuencia, existen otras formas de estudiar los estados emocionales mediante señales fisiológicas y cerebrales, como la respuesta galvánica de la piel (GSR), la electromiografía (EMG), la frecuencia cardíaca (FC), la frecuencia respiratoria (FR), la EEG, la resonancia magnética funcional (fMRI) o la tomografía por emisión de positrones (PET) [2].

La EEG permite medir la actividad eléctrica del cerebro colocando electrodos sobre el cuero cabelludo. Aunque no muestra directamente lo que ocurre en cada neurona y no tiene tanta precisión espacial como otras técnicas, sí permite observar cambios muy rápidos en la actividad cerebral. Trabajar con señales EEG también tiene sus dificultades, ya que se registran con una amplitud muy baja y pueden verse afectadas por distintas fuentes de ruido. Estos ruidos, conocidos como artefactos, pueden producirse por movimientos musculares, movimientos oculares o parpadeos. Además, en el reconocimiento de emociones no basta con obtener una señal y clasificarla directamente, ya que es necesario comprobar que los cambios observados están realmente relacionados con el estado emocional y no con otros factores externos. Por ello la obtención de estas señales debe realizarse en un entorno controlado y acompañarse de técnicas que permitan reducir o eliminar el ruido presente en los datos [2].

El interés de utilizar señales de EEG está en que permiten observar cómo varía la actividad cerebral ante distintos estímulos emocionales. Al registrar estas respuestas, es posible analizar los datos obtenidos y buscar patrones que ayuden a diferenciar unos estados emocionales de otros. Esto permite plantear el reconocimiento de emociones no solo como un problema de clasificación, sino también como una forma de estudiar

qué información de la señal cerebral está relacionada con cada emoción.

1.3. Reconocimiento automático de emociones

El reconocimiento automático de emociones consiste en utilizar datos y modelos computacionales para identificar el estado emocional de una persona. Con ese objetivo, primero se obtiene algún tipo de información del usuario, después se procesan esos datos y finalmente se entrena un modelo capaz de asociar ciertos patrones con una emoción concreta.

Esta información puede obtenerse de distintas formas. Algunos enfoques se basan en señales externas, como la expresión facial o la voz: en el primer caso, se analizan imágenes o vídeos para extraer características del rostro y clasificar la emoción asociada; en el segundo, se estudian características acústicas del habla, como el tono, la intensidad o el ritmo, para relacionarlas con distintos estados emocionales [3]. Otros enfoques utilizan señales fisiológicas o cerebrales, que permiten estudiar respuestas internas del organismo y no dependen únicamente de cómo la persona expresa la emoción hacia el exterior [4, 2].

1.4. IA de caja blanca e interpretabilidad en señales biomédicas

En el análisis de señales biomédicas, obtener una buena métrica no siempre es suficiente. Un modelo puede conseguir una alta precisión, pero si no se entiende en qué información se basa para tomar sus decisiones, resulta más difícil confiar en sus resultados. Esto es especialmente importante en contextos médicos, donde interesa saber si el modelo está utilizando patrones coherentes de la señal o si, por el contrario, está aprendiendo ruido, artefactos o relaciones poco fiables.

Las inteligencias artificiales de caja blanca o explicables son modelos de aprendizaje automático cuyo funcionamiento puede analizarse de forma más directa. A diferencia de los modelos de caja negra, en los que normalmente solo se observan la entrada y la salida, estos modelos permiten entender mejor qué variables han influido en la decisión final. Esto no significa que sean simples o poco útiles, sino que su estructura facilita interpretar cómo se llega a una predicción. Un clasificador que todos conocemos y no lo sabemos es el árbol de decisión, esta “inteligencia” es la que usa el conocido juego

Akinator¹, donde va tomando decisiones a partir de una serie de condiciones. En este tipo de modelos es más fácil seguir el razonamiento que lleva a una determinada salida [5].

Por este motivo, la interpretabilidad tiene un papel importante, no se busca únicamente entrenar un modelo capaz de clasificar emociones, sino también comprender qué información de la señal está utilizando para hacerlo. En el caso de señales de EEG, esto resulta especialmente útil, ya que permite estudiar qué canales, bandas espectrales o características de la señal tienen más peso en la clasificación de una emoción. Esta idea está relacionada con la necesidad de emplear modelos interpretables en decisiones de alto impacto, especialmente en áreas como la salud, donde entender el comportamiento del modelo puede ser tan importante como el resultado obtenido [6].

¹<https://akinator.com/>

Capítulo 2

Objetivos y planificación

En este capítulo se presenta el planteamiento del problema que se quiere abordar, así como los objetivos principales. También se describen los requisitos que debe cumplir la solución desarrollada, la metodología seguida durante el proyecto y la planificación utilizada para organizar las distintas fases del trabajo. De esta forma, se establece una visión general de qué se pretende conseguir y cómo se ha llevado a cabo.

2.1. Descripción y objetivo

Tras haber presentado el contexto general del reconocimiento de emociones mediante señales de EEG, este trabajo se centra en llevar esa idea a un caso concreto. El objetivo es desarrollar un sistema capaz de analizar señales cerebrales y utilizarlas para distinguir entre tres estados emocionales: alegría, tristeza y estado neutro.

La elección de estas tres clases permite comparar una emoción positiva, una emoción negativa y una situación más neutral. A partir de las señales registradas, se pretende aplicar un proceso completo que incluya el tratamiento de los datos, la extracción de información relevante y el entrenamiento de modelos de aprendizaje automático.

El objetivo principal no es únicamente obtener un modelo que clasifique correctamente las emociones, sino también analizar qué información de la señal influye más en esa clasificación. Para ello, se dará importancia a la interpretabilidad de los modelos, con el fin de estudiar qué características de la señales pueden estar más relacionadas con cada estado emocional.

2.2. Requisitos

Para alcanzar los objetivos planteados, el trabajo debe cumplir una serie de requisitos relacionados con el funcionamiento del sistema y con la interpretación de los resultados obtenidos. Por un lado, el sistema debe ser capaz de procesar señales de EEG y entrenar modelos de clasificación. Por otro lado, debe permitir analizar qué información de la señal influye en las decisiones tomadas por dichos modelos.

Requisitos del sistema

- El sistema debe permitir trabajar con señales de EEG asociadas a los estados emocionales considerados en el trabajo: alegría, tristeza y estado neutro.
- Debe permitir organizar los datos tanto de forma general como separando la información correspondiente a cada sujeto.
- Debe incluir una fase de preprocesado de las señales de EEG, con el objetivo de eliminar el ruido y preparar los datos antes de su análisis.
- Debe permitir extraer características relevantes de las señales, teniendo en cuenta la información procedente de distintos canales y bandas espectrales.
- Debe entrenar y evaluar un modelo general utilizando datos de varios sujetos, con el objetivo de analizar el comportamiento global del sistema.
- Debe entrenar y evaluar modelos individuales por sujeto, debido a la variabilidad existente entre las señales de distintas personas.
- Debe permitir comparar los resultados obtenidos entre el enfoque general y el enfoque individual.
- Debe evaluar el rendimiento de los modelos mediante métricas adecuadas para tareas de clasificación.

Requisitos de explicabilidad e interpretación

- El sistema debe permitir analizar qué características de la señal tienen mayor importancia en la clasificación de emociones.

- Debe permitir estudiar la influencia de los distintos canales de EEG utilizados durante el entrenamiento.
- Debe permitir analizar la importancia de las bandas espectrales extraídas de la señal.
- Debe generar una representación topográfica general que ayude a visualizar qué zonas del cuero cabelludo tienen mayor relevancia en el análisis realizado.
- Los resultados obtenidos deben presentarse de forma comprensible, no solo como métricas numéricas, sino también mediante análisis que ayuden a interpretar el comportamiento del modelo.

2.3. Metodología de trabajo

La metodología seguida en este trabajo se ha organizado de forma progresiva, comenzando por una revisión de la literatura relacionada con el reconocimiento de emociones mediante señales de EEG, el preprocesado de este tipo de señales, la extracción de características y el uso de modelos interpretables. Esta revisión permitió entender mejor el problema y definir el enfoque técnico que se iba a seguir.

Una vez situado el contexto, se llevó a cabo un análisis inicial del conjunto de datos proporcionado por el tutor, el cual fue registrado previamente con sujetos reales. En esta fase se estudió la estructura de los datos, los sujetos disponibles, las clases emocionales consideradas y la forma en la que estaban agrupados los registros.

A partir de este análisis, se desarrolló el flujo principal del trabajo. Primero se realizó el preprocesado de las señales de EEG, con el objetivo de preparar los datos antes de su análisis. Después se llevó a cabo la extracción de características, teniendo en cuenta información procedente de distintos canales y bandas espectrales. Estas características fueron utilizadas posteriormente como entrada para los modelos de aprendizaje automático.

Antes del entrenamiento, se aplicaron técnicas de análisis de datos y algoritmos de separabilidad para estudiar si las características extraídas permitían diferenciar entre las emociones consideradas. Esta fase ayudó a comprender mejor la distribución de los datos y a valorar qué información podía ser más útil para la clasificación.

Finalmente, se entrenaron y evaluaron distintos clasificadores. Además de analizar

su rendimiento, también se prestó atención a la interpretabilidad de los resultados, con el objetivo de entender qué características, canales o bandas tenían mayor influencia en las decisiones tomadas por los modelos.

2.4. Planificación y seguimiento

La planificación del trabajo se realizó dividiendo el proyecto en tareas concretas y manejables, siguiendo una metodología inspirada en el método *Getting Things Done*. Esta forma de organización permitió separar el trabajo en bloques claros y avanzar de manera progresiva, evitando abordar todo el proyecto de forma desordenada.

El seguimiento se llevó a cabo de forma semanal mediante tutorías y comunicación por correo electrónico con el tutor. A través de este seguimiento se comentaban los avances realizados, se resolvían dudas y se validaban las decisiones tomadas en cada fase del trabajo. Esto permitió ajustar el planteamiento cuando fue necesario y corregir posibles errores durante el desarrollo.

El proceso seguido en cada bloque fue similar: primero se realizaba una lectura o estudio previo para entender la parte que se iba a trabajar; después, se comentaban con el tutor las ideas aprendidas y el plan de acción; y finalmente se implementaba la solución correspondiente, revisándola en función de los resultados obtenidos y del feedback recibido.

Gracias a esta organización, el proyecto pudo avanzar de forma controlada, manteniendo un seguimiento continuo de las tareas y asegurando que cada decisión técnica estuviera alineada con los objetivos del trabajo.

Capítulo 3

Fundamentos teóricos

En este capítulo se presentan los fundamentos teóricos necesarios para entender el desarrollo del trabajo. Para ello, se introduce primero la señal EEG, la colocación de los electrodos y el análisis por bandas espectrales. Después, se describe su relación con el reconocimiento de emociones y se revisan los conceptos principales de aprendizaje automático, evaluación e interpretabilidad que se emplean en el proyecto.

3.1. Señales EEG y posicionamiento de electrodos

La EEG es una técnica que permite registrar la actividad eléctrica del cerebro mediante electrodos colocados en la cabeza. Lo que se mide realmente no es la actividad de una neurona aislada, sino pequeñas variaciones de voltaje generadas por la actividad conjunta de muchas neuronas, que llegan hasta la superficie de la cabeza y pueden ser captadas por los electrodos.

Estas señales suelen tener una amplitud muy baja, por lo que su registro requiere un equipo específico de adquisición y electrodos distribuidos por distintas zonas del cuero cabelludo. Dependiendo del dispositivo utilizado, los electrodos pueden necesitar gel o pasta conductora para mejorar el contacto con la piel, aunque también existen gorros que emplean electrodos secos o sensores humedecidos con agua mediante pequeñas esponjas.

Durante una grabación, el sujeto normalmente permanece sentado o en reposo mientras se registran las señales, y en estudios de emociones puede ser expuesto a estímulos como imágenes, sonidos o vídeos para observar cómo cambia la actividad cerebral. Aunque no siempre es obligatorio utilizar una sala médica específica, sí es recomendable realizar la grabación en un entorno controlado, cómodo y con las menores distracciones posibles, ya que factores como el movimiento, los parpadeos, la actividad

muscular, la iluminación o la incomodidad pueden introducir ruido en la señal. Por este motivo, el registro no consiste únicamente en colocar electrodos y guardar datos, sino en intentar obtener una señal lo suficientemente limpia y controlada como para que posteriormente pueda analizarse y relacionarse con el estado mental o emocional del sujeto [2].

La corteza cerebral es una de las partes principales del cerebro y suele dividirse en cuatro lóbulos: frontal, temporal, parietal y occipital. Cada uno de ellos está relacionado con funciones diferentes. El lóbulo frontal participa en procesos asociados al pensamiento consciente; el temporal está relacionado con sentidos como el oído y el olfato, además del procesamiento de estímulos complejos como caras o escenas; el parietal integra información procedente de distintos sentidos y participa en la manipulación de objetos; y el occipital está principalmente vinculado con la visión [2]. Aunque la grabación no permita localizar con precisión exacta la actividad de cada región cerebral, la posición de los electrodos ofrece una referencia aproximada de la zona desde la que se está registrando la señal.

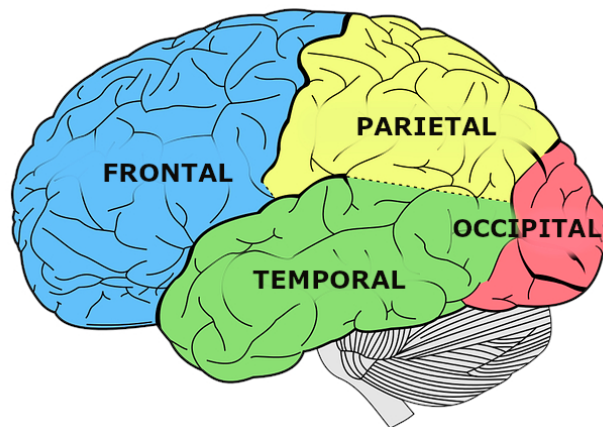


Figura 3.1: Principales lóbulos de la corteza cerebral. Fuente: [7].

La colocación de los electrodos no se realiza de forma aleatoria, sino siguiendo sistemas estandarizados que permiten registrar la actividad cerebral de manera ordenada y comparable entre sujetos. Uno de los sistemas más utilizados es el sistema internacional 10-20, en el que las posiciones de los electrodos se calculan a partir de referencias anatómicas de la cabeza, como el nasion, el inion y los puntos preauriculares. El nombre 10-20 hace referencia a que las distancias entre posiciones se establecen como porcentajes del tamaño de la cabeza, lo que permite adaptar el montaje a cada persona. Además, existen sistemas más densos, como el sistema 10-10 o el montaje estándar 10-05, que añaden posiciones intermedias y permiten trabajar con configuraciones de

mayor número de electrodos. En la práctica, los registros de EEG pueden realizarse con montajes de distinta densidad, como 32 o 64 electrodos. Un montaje de 32 canales puede ofrecer una cobertura general, mientras que uno de 64 canales permite recoger información con mayor detalle espacial. [8, 9].

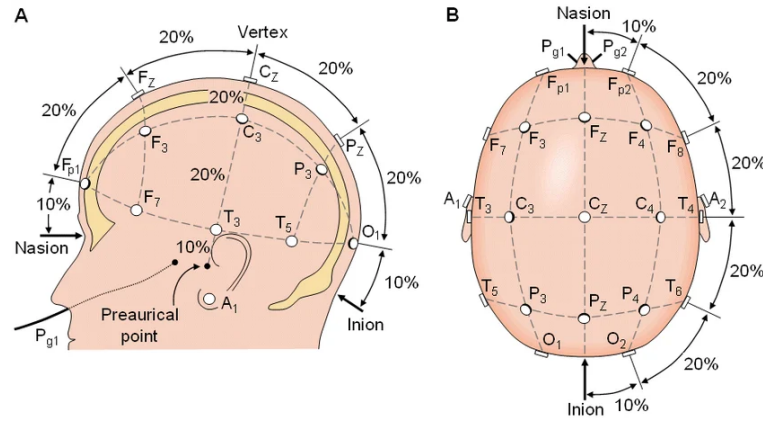


Figura 3.2: Sistema internacional 10-20 para la colocación de electrodos EEG. Fuente: [10].

La nomenclatura de los electrodos indica la zona aproximada en la que se encuentran: F a la frontal, C a la central, T a la temporal, P a la parietal y O a la occipital. Además, la letra z indica posiciones situadas en la línea media, los números impares suelen corresponder al hemisferio izquierdo y los pares al hemisferio derecho. Esta localización es importante porque cada canal recoge información de una zona concreta, por lo que la posición de los electrodos influye directamente en el análisis posterior, especialmente cuando se estudia qué canales o regiones tienen más importancia en la clasificación de emociones.

3.2. Bandas espectrales

La señal de EEG puede analizarse tanto en el dominio temporal como en el dominio frecuencial. En este segundo caso, la señal se descompone en distintos rangos de frecuencia, conocidos como bandas espectrales. Estas bandas permiten estudiar qué parte de la actividad cerebral aparece con mayor presencia en cada momento y facilitan la extracción de características para su posterior uso en modelos de aprendizaje automático. Aunque los límites exactos pueden variar ligeramente según el autor, normalmente se distinguen cinco bandas principales: delta, theta, alpha, beta y gamma [2].

Banda	Rango aproximado	Asociación general
Delta	1–4 Hz	Se suele relacionar con estados de sueño profundo o baja actividad consciente.
Theta	4–8 Hz	Se asocia a estados de somnolencia, relajación o actividad subconsciente.
Alpha	8–12 Hz	Suele aparecer en estados de relajación, especialmente cuando la persona está despierta pero tranquila.
Beta	12–30 Hz	Se relaciona con estados de mayor actividad mental, atención o concentración.
Gamma	>30 Hz	Se asocia con una actividad cerebral más intensa o procesos de integración de información.

Tabla 3.1: Bandas espectrales principales de la señal EEG.

El análisis por bandas es importante porque muchas veces la información relevante de una señal de EEG no se encuentra directamente en la señal completa, sino en cómo cambia la actividad dentro de determinados rangos de frecuencia. Por ejemplo, en lugar de utilizar únicamente la señal cruda, es habitual calcular características como la potencia de cada banda en distintos canales. De esta forma, el modelo puede trabajar con una representación más compacta y útil de la actividad cerebral.

En el reconocimiento de emociones, estas bandas pueden aportar información relevante, ya que la actividad alpha y la asimetría entre hemisferios se han relacionado con la valencia emocional, mientras que las bandas beta y gamma también aparecen asociadas a la diferenciación de ciertos estados afectivos [2]. Por este motivo, en trabajos de clasificación emocional con EEG es frecuente extraer características a partir de varias bandas espectrales, ya que cada una puede aportar información distinta sobre la respuesta cerebral.

3.3. Emociones y actividad cerebral: alegría, tristeza y estado neutro

Las emociones pueden representarse de distintas formas. Una de ellas es mediante modelos discretos, donde cada emoción se trata como una categoría concreta, por ejemplo alegría, tristeza, miedo o ira. Otra forma habitual es utilizar modelos dimensionales, en los que las emociones se describen a partir de ejes continuos. Uno de

los modelos más utilizados es el de valencia-excitación, donde la valencia indica si la emoción es más positiva o negativa, mientras que la excitación representa el nivel de activación asociado a esa emoción. Desde esta perspectiva, la alegría puede situarse en una zona de valencia positiva, la tristeza en una zona de valencia negativa y el estado neutro en una posición más intermedia, con una carga emocional menos marcada [2].

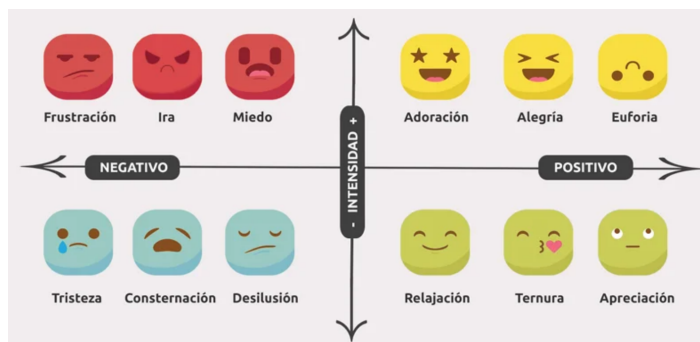


Figura 3.3: Representación simplificada de emociones según valencia y arousal. Adaptado de [11].

Para detectar estados emocionales a partir de señales de EEG, los estudios analizan cómo varía la actividad cerebral en distintas regiones ante diferentes estímulos, siendo las zonas frontal y parietal las que aparecen con mayor frecuencia como relevantes [2]. A partir de estas observaciones, se extraen características de la señal que se utilizan como entrada a modelos de clasificación. Sin embargo, estas relaciones no constituyen patrones universales, sino tendencias que varían considerablemente entre personas: la respuesta cerebral ante una misma emoción puede diferir de forma notable de un sujeto a otro. Esta variabilidad intersujeto es uno de los principales retos del campo y explica por qué los modelos entrenados con datos de varios sujetos no siempre generalizan bien, lo que justifica también el análisis de modelos individuales por sujeto [12].

De forma general, las emociones positivas suelen asociarse con una mayor actividad relativa en regiones frontales izquierdas, mientras que las emociones negativas se han relacionado con una mayor actividad relativa en regiones frontales derechas. Esta diferencia entre hemisferios se conoce como asimetría frontal y se ha utilizado en distintos estudios como una posible señal relacionada con la valencia emocional. El estado neutro no debería presentar una activación emocional tan marcada, por lo que puede utilizarse como referencia frente a las emociones con mayor carga afectiva [2].

3.4. Aprendizaje automático aplicado a EEG

El aprendizaje automático permite construir modelos capaces de aprender patrones a partir de datos y utilizarlos posteriormente para realizar predicciones sobre nuevos ejemplos. En una tarea de clasificación supervisada, cada muestra de entrada se asocia a una etiqueta conocida durante el entrenamiento. A partir de estos ejemplos, el modelo aprende una relación entre las variables de entrada y la clase correspondiente, con el objetivo de aplicar ese conocimiento a datos no vistos previamente [13].

En el caso de señales de EEG, la entrada del modelo no suele ser la señal completa registrada en el tiempo, ya que esta puede contener ruido, redundancia y una dimensionalidad elevada. Por ello, normalmente se trabaja con una representación basada en características, es decir, con variables numéricas que resumen información relevante de la señal. Estas características pueden describir aspectos temporales, frecuenciales o espaciales de la actividad cerebral, dependiendo del tipo de análisis realizado.

Desde el punto de vista del clasificador, cada muestra queda representada por un conjunto de características y una etiqueta de clase. El modelo intenta encontrar relaciones entre esas características y la clase asociada, de forma que pueda asignar una etiqueta a nuevas muestras. Si las características contienen información discriminativa, las muestras de una misma clase tenderán a presentar cierta similitud entre ellas, mientras que las muestras de clases distintas deberían mostrar diferencias aprovechables por el modelo [14].

Sin embargo, en señales de EEG este proceso no es directo. La actividad cerebral registrada en superficie es compleja, puede verse afectada por artefactos y presenta una variabilidad considerable entre personas. Como consecuencia, dos muestras asociadas a una misma emoción no tienen por qué ser muy parecidas, y dos emociones distintas pueden generar patrones parcialmente solapados. Esta dificultad hace que la evaluación del modelo sea especialmente importante, ya que no basta con que aprenda bien los ejemplos de entrenamiento: debe comprobarse si mantiene un comportamiento adecuado ante datos que no ha utilizado para ajustar sus parámetros.

Además, en señales biomédicas como la EEG, el rendimiento del modelo no es el único aspecto relevante. También interesa analizar qué información está utilizando para tomar sus decisiones y si esa información resulta coherente con el problema estudiado. Por ello, antes de describir los clasificadores utilizados, conviene diferenciar

entre modelos interpretables y modelos de caja negra, ya que esta distinción condiciona la forma en la que pueden analizarse los resultados obtenidos.

3.4.1. Modelos interpretables frente a modelos de caja negra

Un modelo de caja negra es aquel cuyo funcionamiento interno no resulta fácil de seguir por una persona. Aunque se conocen las características que recibe como entrada y la predicción que genera como salida, no siempre es sencillo entender cómo transforma esas variables en una decisión concreta. Esto suele ocurrir en modelos con una estructura compleja, como redes neuronales profundas¹ o métodos de conjunto como Random Forest², donde la relación entre entrada y salida queda distribuida entre numerosos parámetros o transformaciones internas [5]. Un ejemplo conocido de sistema basado en modelos de este tipo es ChatGPT³: el usuario puede observar el texto que introduce y la respuesta que recibe, pero el proceso interno que genera esa respuesta no se presenta como una secuencia de reglas fácilmente interpretable.

Frente a estos modelos, los modelos interpretables o de caja blanca tienen una estructura que permite comprender de forma más directa cómo se obtiene una predicción. La interpretación no se añade únicamente después de entrenar el modelo, sino que forma parte de su propio funcionamiento: sus parámetros, reglas o condiciones pueden examinarse y relacionarse con las variables de entrada. Por ejemplo, en un modelo lineal se puede estudiar el peso asociado a cada característica, mientras que en un sistema basado en reglas se puede seguir qué condiciones se han cumplido para llegar a una salida concreta. Esta transparencia facilita analizar qué variables intervienen en el resultado, si su influencia es razonable y si el comportamiento del modelo coincide con lo esperado en el problema estudiado [5].

La elección entre modelos interpretables y modelos de caja negra suele plantear un compromiso entre capacidad de representación e interpretabilidad. Los modelos más complejos pueden capturar relaciones no lineales difíciles de describir de forma simple, pero sus decisiones suelen ser menos transparentes. En cambio, los modelos interpretables pueden ser más fáciles de analizar, aunque no siempre alcanzan el mismo rendimiento en todos los problemas. No obstante, este compromiso no debe entenderse como una regla absoluta: en contextos donde la explicación del resultado es

¹[https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)](https://en.wikipedia.org/wiki/Neural_network_(machine_learning))

²https://en.wikipedia.org/wiki/Random_forest

³<https://openai.com/chatgpt/>

importante, es recomendable valorar primero si un modelo interpretable puede ofrecer un comportamiento adecuado antes de recurrir a modelos más opacos [6].

3.4.2. Clasificadores

En este trabajo se consideran dos clasificadores interpretables: regresión logística⁴ y árbol de decisión⁵. Ambos parten de la misma idea general: reciben una muestra representada por sus características y asignan una clase de salida. Sin embargo, cada uno aprende esa relación de una forma distinta.

Regresión logística. La regresión logística es un modelo lineal de clasificación. Aunque su nombre pueda sugerir una tarea de regresión, se utiliza para estimar la probabilidad de pertenecer a una clase. Para ello combina las características de entrada mediante una suma ponderada y transforma el resultado en una probabilidad. En el caso binario, esta idea puede expresarse como:

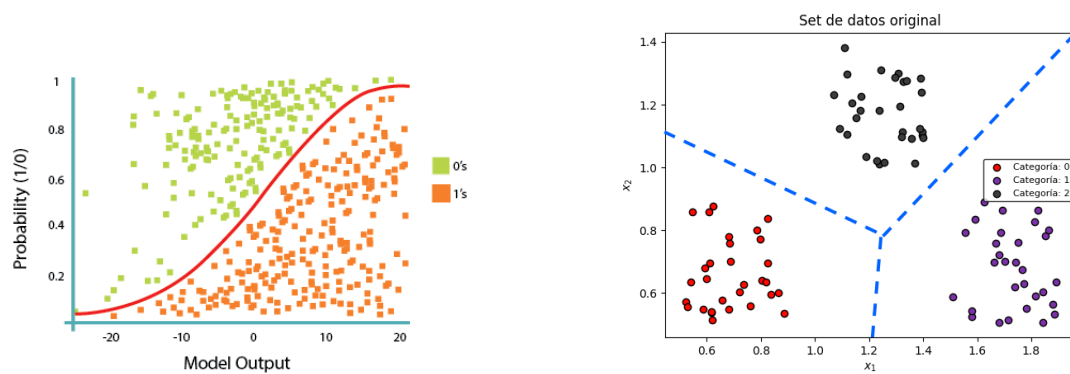
$$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

Ecuación 3.1: Probabilidad estimada por una regresión logística binaria.

donde $\mathbf{x}$ representa las características de la muestra, $\mathbf{w}$ los pesos aprendidos por el modelo, b el sesgo y σ la función logística, que convierte la puntuación del modelo en un valor entre 0 y 1. Esta formulación corresponde al caso binario, en el que solo existen dos clases posibles. En problemas con tres o más clases, como ocurre al distinguir entre varias emociones, la regresión logística se extiende calculando una puntuación para cada clase y seleccionando aquella con mayor probabilidad [13].

⁴https://en.wikipedia.org/wiki/Logistic_regression

⁵https://en.wikipedia.org/wiki/Decision_tree_learning



(a) Regresión logística binaria. Fuente: [15].

(b) Regresión logística multiclase. Fuente: [16].

Figura 3.4: Comparación entre regresión logística binaria y multiclase.

Árbol de decisión. Un árbol de decisión clasifica las muestras mediante una secuencia de reglas. Su estructura está formada por nodos, ramas y hojas: en cada nodo se evalúa una condición sobre una característica, cada rama representa el resultado de esa condición y cada hoja contiene la clase asignada. Por ejemplo, el modelo puede dividir primero las muestras según el valor de una característica concreta y, después, seguir refinando la decisión con nuevas condiciones.

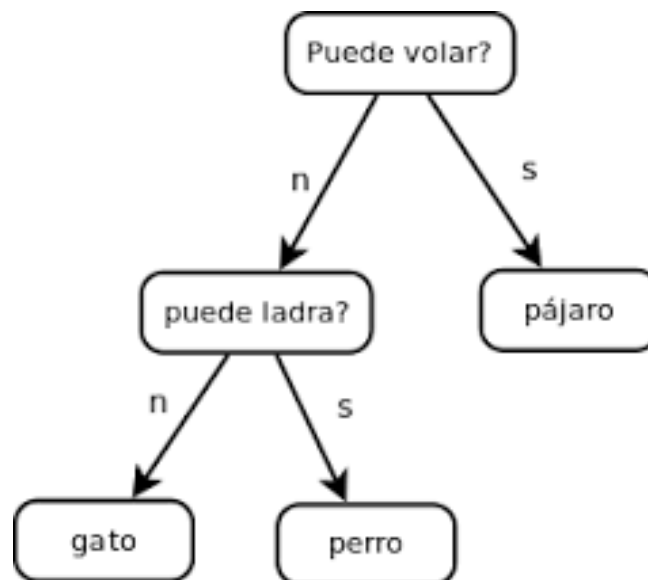


Figura 3.5: Ejemplo de estructura de árbol de decisión. Fuente: [17].

Esta forma de funcionamiento hace que el modelo sea fácil de recorrer e interpretar, ya que la predicción puede explicarse como un camino desde la raíz del árbol hasta una hoja final. Además, a diferencia de un modelo estrictamente lineal, un árbol puede

capturar relaciones no lineales entre características sin exigir una transformación previa de los datos. Su principal limitación es que, si crece demasiado, puede ajustarse en exceso al conjunto de entrenamiento y perder capacidad de generalización [5].

3.4.3. Estrategia de evaluación y validación cruzada

La evaluación de un clasificador debe estimar cómo se comportará ante datos que no ha utilizado durante el entrenamiento. Si solo se mide el rendimiento sobre los mismos ejemplos con los que se ajusta el modelo, el resultado puede ser demasiado optimista, ya que el modelo puede haber aprendido particularidades del conjunto de entrenamiento en lugar de patrones generales. Por ello, en aprendizaje supervisado se distingue entre el ajuste del modelo y la estimación de su capacidad de generalización.

La forma más básica de plantear esta evaluación consiste en separar los datos en dos partes: un conjunto de entrenamiento, utilizado para ajustar el modelo, y un conjunto de prueba, reservado para medir su comportamiento sobre ejemplos no vistos. Esta separación permite simular la situación real en la que el clasificador recibe una nueva muestra y debe asignarle una clase sin haberla usado previamente para aprender.

La validación cruzada *k-fold*⁶ extiende esta idea de separación entre entrenamiento y prueba. En lugar de realizar una única división, el conjunto de datos se divide en k particiones o *folds*. En cada iteración, el modelo se entrena con $k - 1$ particiones y se evalúa con la partición restante. Al repetir el proceso hasta que todos los *folds* han actuado como conjunto de prueba, se obtiene una estimación media del comportamiento del modelo y de su variabilidad entre particiones [18].

⁶[https://en.wikipedia.org/wiki/Cross-validation_\(statistics\)](https://en.wikipedia.org/wiki/Cross-validation_(statistics))

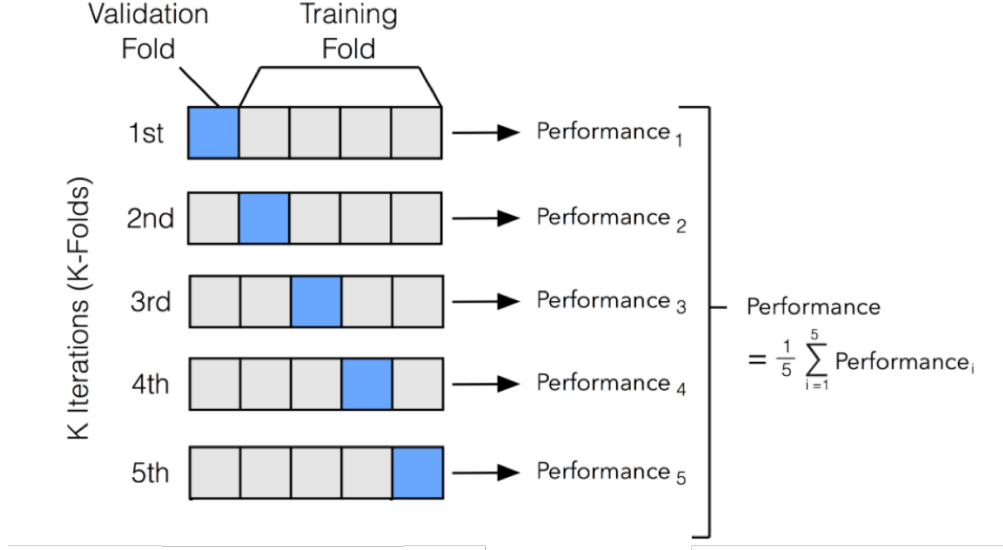


Figura 3.6: Esquema de validación cruzada k -fold. Fuente: [19].

$$\overline{M}_{CV} = \frac{1}{k} \sum_{i=1}^k M_i$$

Ecuación 3.2: Promedio de una métrica en validación cruzada k -fold.

donde $\overline{M}_{CV}$ representa el rendimiento medio estimado mediante validación cruzada, k es el número de particiones y M_i es el valor de la métrica obtenido en el $fold$ i . Esta expresión resume la idea de evaluar el modelo varias veces y combinar los resultados en una medida global.

En problemas de clasificación con varias clases, como el reconocimiento de emociones, es importante que cada partición mantenga una distribución de clases similar a la del conjunto completo. La validación cruzada estratificada busca precisamente conservar proporciones parecidas de cada clase en los distintos $fold$ s. De esta forma se reduce el riesgo de evaluar el modelo en particiones poco representativas, por ejemplo con muy pocas muestras de una emoción concreta [18].

En señales de EEG, además de la distribución de clases, debe tenerse en cuenta la dependencia entre muestras de un mismo sujeto. Si segmentos o muestras procedentes de la misma persona aparecen simultáneamente en entrenamiento y prueba, el modelo puede aprovechar características propias del sujeto en lugar de aprender patrones que generalicen a personas no vistas. Este problema puede producir estimaciones de

rendimiento artificialmente altas, por lo que en estudios con múltiples muestras por individuo es recomendable que la separación entre entrenamiento y prueba respete la agrupación por sujeto [20].

Esta idea está relacionada con la fuga de información, o *data leakage*⁷, que ocurre cuando durante el entrenamiento o la selección del modelo se utiliza información que no estaría disponible en una predicción real. En este contexto no solo importa cómo se dividen los datos, sino también cómo se aplican los pasos previos al modelo. Transformaciones como la normalización, la selección de características o el ajuste de configuraciones deben calcularse utilizando únicamente la parte de entrenamiento de cada partición y aplicarse después sobre la parte de prueba [21].

La validación cruzada también puede utilizarse para comparar distintas configuraciones de un modelo y seleccionar aquella que obtiene mejor comportamiento medio. Sin embargo, esta selección debe separarse conceptualmente de la evaluación final, ya que usar los mismos resultados para escoger el modelo y estimar su rendimiento puede introducir una estimación demasiado optimista del error. Por tanto, la estrategia de evaluación debe diseñarse de forma que mida la capacidad de generalización del modelo y no solo su adaptación a una partición concreta de los datos.

3.4.4. Métricas

Una vez definido cómo se separan los datos para entrenar y evaluar el modelo, es necesario escoger métricas que resuman la relación entre las etiquetas reales y las etiquetas predichas. En un problema de clasificación multiclase, como el reconocimiento de emociones, estas métricas no solo indican cuántas muestras se han clasificado correctamente, sino también qué clases se confunden entre sí y si el rendimiento es equilibrado entre emociones. Las medidas descritas a continuación son habituales en tareas de clasificación y están disponibles en bibliotecas de aprendizaje automático como scikit-learn [22].

- **Matriz de confusión.** La matriz de confusión organiza las predicciones comparando, para cada muestra, la clase real con la clase asignada por el modelo. En el caso binario, esta matriz permite distinguir cuatro situaciones: los verdaderos positivos (TP), que son muestras positivas clasificadas correctamente; los verdaderos negativos (TN), que son muestras negativas clasificadas

⁷[https://en.wikipedia.org/wiki/Leakage_\(machine_learning\)](https://en.wikipedia.org/wiki/Leakage_(machine_learning))

correctamente; los falsos positivos (FP), que son muestras negativas clasificadas erróneamente como positivas; y los falsos negativos (FN), que son muestras positivas clasificadas erróneamente como negativas [23].

		Actual Condition		
		FALSE	TRUE	
Predicted Condition	FALSE	TN	FN	Predicted Negative
	TRUE	FP	TP	Predicted Positive
		Actual Negative	Actual Positive	

Figura 3.7: Ejemplo de matriz de confusión binaria. Fuente: [23].

En clasificación multiclase, la idea se extiende a una matriz con tantas filas y columnas como clases tenga el problema. Las filas representan normalmente las clases reales y las columnas las clases predichas. Los valores de la diagonal principal corresponden a aciertos, mientras que los valores fuera de la diagonal muestran errores entre clases concretas. Esta representación es especialmente útil cuando interesa saber, por ejemplo, si dos emociones tienden a confundirse más que el resto [22].

- **Exactitud o *accuracy*.** La exactitud mide la proporción de muestras clasificadas correctamente respecto al total. Es una métrica sencilla y fácil de interpretar, pero puede ocultar problemas cuando las clases no están equilibradas, ya que un modelo puede obtener una exactitud alta favoreciendo a la clase más frecuente. En clasificación multiclase puede expresarse a partir de la matriz de confusión:

$$\text{Accuracy} = \frac{\sum_{c=1}^C n_{cc}}{\sum_{i=1}^C \sum_{j=1}^C n_{ij}}$$

Ecuación 3.3: Exactitud en clasificación multiclase a partir de la matriz de confusión.

donde C es el número de clases, n_{cc} representa los aciertos de la clase c y n_{ij} indica el número de muestras cuya clase real es i y cuya clase predicha es j .

- **Precisión, recall y F1-score.** Para analizar el comportamiento por clase se pueden calcular métricas considerando cada emoción frente al resto. La precisión mide la fiabilidad de las predicciones realizadas para una clase: de todas las muestras que el modelo asigna a esa clase, indica qué proporción es correcta. El *recall*, también llamado sensibilidad, mide la capacidad del modelo para detectar una clase: de todas las muestras que realmente pertenecen a esa clase, indica qué proporción ha sido clasificada correctamente. El F1-score combina ambas medidas mediante una media armónica, por lo que penaliza los casos en los que una de las dos es baja [22].

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad F1_c = 2 \cdot \frac{P_c \cdot R_c}{P_c + R_c}$$

Ecuación 3.4: Precisión, recall y F1-score para una clase c .

donde P_c es la precisión de la clase c , R_c es su *recall*, TP_c son los verdaderos positivos, FP_c los falsos positivos y FN_c los falsos negativos de esa clase.

- **F1 macro.** En problemas con varias clases, el F1-score puede calcularse para cada clase y después promediarse. El promedio macro asigna el mismo peso a todas las clases, independientemente del número de muestras que tenga cada una. Por ello resulta adecuado cuando se quiere valorar si el modelo funciona de forma equilibrada entre las distintas emociones, sin que una clase con más ejemplos domine el resultado global [22].

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F1_c$$

Ecuación 3.5: F1 macro como promedio no ponderado del F1-score por clase.

- **Media y desviación entre particiones.** Cuando las métricas se calculan dentro de una validación cruzada, cada *fold* produce un valor distinto. La media resume el rendimiento esperado del modelo y la desviación estándar indica la variabilidad entre particiones. Una desviación elevada puede señalar que el resultado depende mucho de qué sujetos o muestras quedan en cada partición, por lo que aporta información complementaria al valor medio.

3.5. Técnicas de explicación

Para analizar el comportamiento de los modelos entrenados, en este trabajo se emplean dos tipos de técnicas de explicación: los valores SHAP y los mapas topográficos. Ambas permiten examinar, desde perspectivas distintas, qué información de la señal de EEG tiene más peso en las decisiones del clasificador.

Los mapas topográficos, o *topomaps*, permiten representar información de la señal de EEG sobre una distribución espacial similar a la posición real de los electrodos en el cráneo. De esta forma, es posible visualizar qué zonas presentan mayor actividad o mayor importancia dentro del análisis realizado. En el reconocimiento de emociones, es habitual generar un mapa por cada banda espectral analizada, ya que la importancia de cada región puede variar en función de la frecuencia. Además, los valores representados pueden interpretarse en valor absoluto, indicando la magnitud de la contribución de cada zona, o con signo, mostrando si esa contribución empuja hacia una clase o la aleja. Los *topomaps* permiten comparar visualmente los patrones asociados a alegría, tristeza y estado neutro a lo largo de las distintas bandas espectrales.

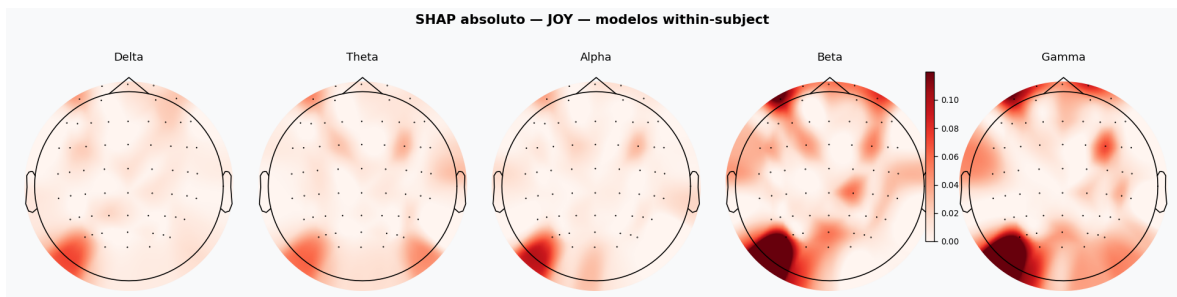


Figura 3.8: Ejemplo de topomaps por banda espectral para la clase alegría.

Por otro lado, SHAP es una técnica de explicación basada en la teoría de juegos cooperativos que asigna a cada característica un valor según cuánto contribuye a la predicción del modelo. Su principal ventaja es que ofrece explicaciones tanto a nivel global, mostrando qué variables son más relevantes en general, como a nivel individual, indicando en qué dirección empuja cada característica una predicción concreta. En modelos lineales, este valor puede calcularse de forma exacta a partir de los coeficientes del modelo, sin necesidad de aproximaciones [24].

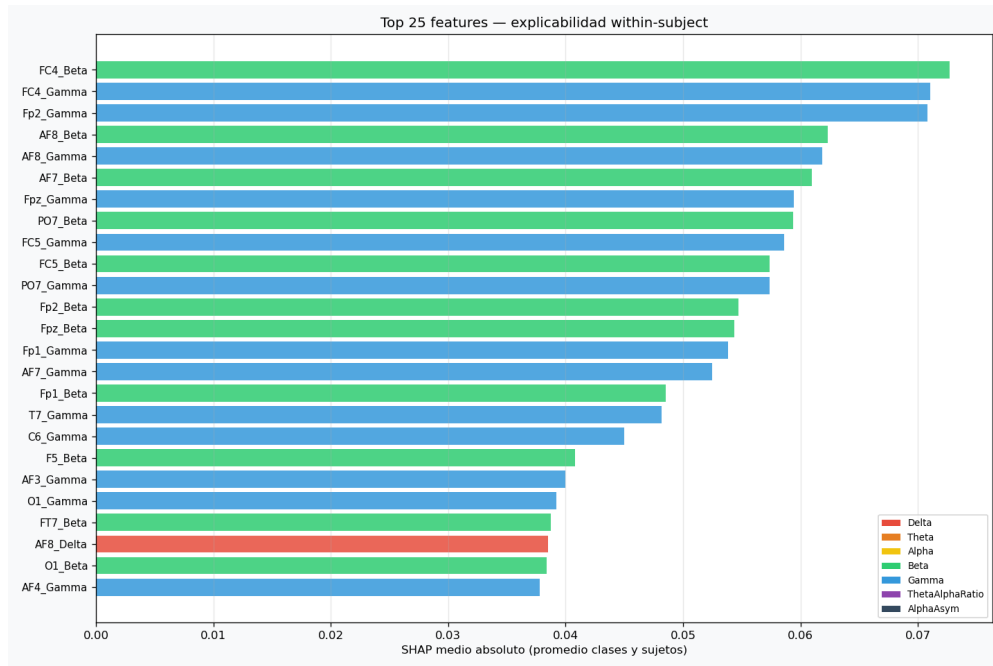


Figura 3.9: Ejemplo de importancia de características mediante valores SHAP.

Capítulo 4

Materiales, datos y entorno de trabajo

En este capítulo se describen los materiales utilizados para el desarrollo del proyecto y la forma en la que se organizaron los datos de partida. También se presenta el entorno físico y software empleado, así como las principales librerías utilizadas durante el procesamiento, entrenamiento y análisis de los resultados.

4.1. Protocolo experimental y estímulos auditivos

Las señales utilizadas en este trabajo fueron registradas en la Facultad de Medicina de la Universidad Complutense de Madrid, en un entorno preparado para la adquisición de señales fisiológicas. Las sesiones se realizaron en una sala controlada, sin iluminación y acondicionada para reducir tanto el ruido acústico externo como posibles interferencias electromagnéticas. Estas condiciones son importantes en registros de EEG, ya que la señal tiene una amplitud muy baja y puede verse afectada por movimientos, distracciones o ruido procedente del entorno.

Cada sesión de registro tuvo una duración aproximada de hora y media. Durante la grabación, el participante permanecía en la sala mientras se le presentaban distintos estímulos auditivos diseñados para inducir estados emocionales concretos. Para ello se utilizaron frases sin significado léxico relevante, de forma que la emoción no dependiera tanto del contenido semántico de la frase, sino de la prosodia¹, es decir, de elementos como la entonación, el ritmo, la intensidad o la forma de pronunciarla.

Los estímulos auditivos se asociaban a las condiciones emocionales consideradas: alegría, tristeza y estado neutro. La elección de este tipo de estímulos permite centrar

¹[https://es.wikipedia.org/wiki/Prosodia_\(ling%C3%BC%C3%ADstica\)](https://es.wikipedia.org/wiki/Prosodia_(ling%C3%BC%C3%ADstica))

la respuesta emocional en la forma en la que se escucha la frase, evitando en parte que el significado de las palabras condicione la interpretación del sujeto. De esta manera, las grabaciones obtenidas contienen segmentos de EEG relacionados con cada una de las condiciones emocionales que posteriormente se emplean en el proceso de clasificación.

Para la adquisición se utilizó un sistema de EEG con un gorro de electrodos humedecidos, lo que mejora el contacto entre los sensores y el cuero cabelludo. Antes de comenzar la grabación era necesario colocar correctamente el gorro, revisar el contacto de los electrodos y preparar al participante para reducir movimientos innecesarios durante la sesión.



(a) Sala de grabación utilizada para la adquisición de las señales.



(b) Sistema EEG con gorro de electrodos humedecidos.

Figura 4.1: Entorno y material utilizados durante el registro de las señales EEG.

4.2. Descripción general de los datos

El conjunto de datos de partida está formado por registros crudos de EEG almacenados en formato *European Data Format* (EDF)², antes de aplicar cualquier etapa de preprocesado, limpieza o extracción de características. Participaron 24 sujetos y, para cada uno, se realizó una grabación por cada emoción considerada (Joy, Neutro

²<https://www.edfplus.info/specs/edf.html>

y Sad). En consecuencia, el conjunto inicial está compuesto por 72 archivos crudos.

Los participantes aparecen identificados mediante códigos anónimos, evitando incluir información personal en los nombres de archivo. Cada archivo sigue una nomenclatura que permite reconocer el sujeto, el número de registro y la condición emocional, por ejemplo 211-000531-01_JOY.edf. En este caso, 211-000531 identifica al sujeto, 01 al registro concreto y JOY a la condición emocional.

Cada archivo en formato EDF contiene las señales registradas durante una condición experimental concreta. Las cabeceras de los archivos incluyen 66 señales: 64 canales de registro y dos canales auxiliares. El canal **STIM**, abreviatura de *stimulation*, almacena los marcadores de eventos, es decir, las marcas que indican en qué momentos aparecen estímulos o segmentos relevantes de la grabación. El canal **SYNC** se utiliza como referencia temporal para sincronizar esos eventos con la señal registrada. Estos eventos son importantes porque permiten localizar dentro del registro las partes que contienen la información válida para el análisis posterior.

4.3. Organización del repositorio

El repositorio se organizó en cuatro bloques principales: datos, código, modelos y resultados. A nivel general, la estructura principal es la siguiente:

```
TFG/
|-- data/
|-- scripts/
|-- models/
'-- results/
```

La carpeta **data/** reúne los datos utilizados y los archivos generados durante las primeras etapas del flujo de trabajo:

```
data/
|-- raw/
|   |-- edf/
|   |   |-- Joy/
|   |   |-- Neutro/
|   |   '--- Sad/
|   |-- audios/
|   |-- protocolos/
|   '--- referencia/
|-- processed/
|   |-- epochs/
```

```
|  '-- stats/
'-- features/
```

- **raw/**: contiene los datos originales sin modificar, incluyendo registros EDF, audios, protocolos y archivos de referencia.
- **raw/edf/**: organiza los registros crudos según la emoción asociada: alegría, estado neutro y tristeza.
- **processed/**: almacena los datos obtenidos tras el preprocesado, como los segmentos extraídos de las señales y estadísticas asociadas.
- **features/**: guarda las matrices de características, las etiquetas y la información auxiliar utilizada para entrenar los modelos.

La carpeta **scripts/** contiene el código utilizado durante el desarrollo del proyecto. Las carpetas numeradas siguen el orden general del proceso:

```
scripts/
|-- 0_install/
|-- 1_preprocesado/
|-- 2_features/
|-- 3_train/
|-- 4_separability/
'-- 5_explicability/
```

- **0_install/**: incluye archivos relacionados con la preparación inicial del entorno.
- **1_preprocesado/**: transforma los registros crudos en datos segmentados y preparados para el análisis.
- **2_features/**: extrae las características utilizadas posteriormente por los clasificadores.
- **3_train/**: agrupa el entrenamiento de modelos globales y modelos por sujeto.
- **4_separability/**: estudia la separación entre clases o sujetos en el espacio de características.
- **5_explicability/**: genera explicaciones basadas en SHAP, mapas topográficos y visualizaciones de los modelos interpretables.

La carpeta `models/` almacena los modelos generados durante los experimentos:

```
models/
|-- all_subjects/
|   |-- logReg/
|   '-- decisionTree/
'-- within_subject/
    |-- logReg/
    '-- decisionTree/
```

- `all_subjects/`: contiene los modelos entrenados utilizando información conjunta de varios sujetos, separados según el clasificador empleado.
- `within_subject/`: guarda los modelos entrenados de forma individual para cada sujeto, también organizados por clasificador.
- `logReg/` y `decisionTree/`: separan los modelos correspondientes a regresión logística y árbol de decisión.

Por último, la carpeta `results/` reúne las salidas generadas durante la evaluación y el análisis de los modelos:

```
results/
|-- all_subjects/
|   |-- logReg/
|   '-- decisionTree/
'-- within_subject/
    |-- logReg/
    '-- decisionTree/
|-- checks/
|-- separability/
'-- explicability/
```

- `all_subjects/`: contiene métricas, gráficas y salidas asociadas a los experimentos con modelos globales.
- `within_subject/`: reúne los resultados obtenidos en los experimentos realizados de forma individual por sujeto.
- `logReg/` y `decisionTree/`: separan las figuras y salidas específicas de cada clasificador.

- **checks/**: almacena las salidas generadas por la comprobación de sujetos válidos, como balances de épocas, listas de sujetos aceptados o descartados y gráficas de control espectral.
- **separability/**: guarda los análisis de separación entre clases o sujetos.
- **explicability/**: contiene los resultados de interpretabilidad, como importancias de características, mapas topográficos, coeficientes de la regresión logística y visualizaciones de árboles de decisión.

4.4. Entorno de trabajo

El proyecto se desarrolló en un entorno local organizado de forma que el procesamiento de señales, la extracción de características, el entrenamiento de modelos y la generación de resultados pudieran ejecutarse desde el mismo repositorio. El código y los archivos de instalación se encuentran disponibles en GitHub³, lo que facilita consultar la estructura completa del proyecto y reproducir el entorno utilizado.

4.4.1. Entorno físico y software

La configuración software se apoyó principalmente en dos elementos:

- **Python**⁴: es el lenguaje de programación utilizado para implementar el flujo de trabajo del proyecto, desde el preprocesado de las señales hasta la extracción de características, el entrenamiento de modelos y la generación de resultados.
- **Conda**⁵: es un gestor de paquetes y entornos que permite instalar librerías concretas y aislarlas del resto del sistema.

En este trabajo se empleó Python 3.11.14 y Conda se instaló mediante Miniconda, una distribución ligera que proporciona las herramientas necesarias sin incluir de partida un gran número de paquetes adicionales. La instalación se centralizó en la carpeta `scripts/0_install/`, donde se incluye un archivo que crea o actualiza el entorno con las versiones empleadas durante el desarrollo. Los canales utilizados

³<https://github.com/LiamSaboridoSueiro/TFG>

⁴<https://www.python.org/doc/>

⁵<https://docs.conda.io/>

para construirlo fueron `defaults`, el canal principal de paquetes de Anaconda, y `conda-forge`, un canal comunitario que ofrece muchas librerías científicas y versiones actualizadas.

Aunque el proyecto puede reproducirse en cualquier ordenador con un entorno Python equivalente, el tiempo de ejecución de algunas etapas puede variar en función del procesador, la memoria disponible y el número de archivos procesados. En este trabajo se utilizó como equipo principal un portátil con sistema operativo Ubuntu 24.04 LTS⁶ sobre arquitectura `x86_64`, procesador Intel Core i7-10750H, 16 GB de memoria RAM y gráfica NVIDIA⁷ GeForce GTX 1650 Ti Mobile. No obstante, el flujo principal del proyecto se basa en procesamiento de señal y modelos clásicos de aprendizaje automático, por lo que no requiere necesariamente una *Graphics processing unit* (GPU) dedicada.

4.4.2. Librerías y dependencias principales

Las librerías principales del entorno de trabajo fueron las siguientes:

- **MNE-Python**⁸: librería especializada en el análisis de señales neurofisiológicas, especialmente EEG y magnetoencefalografía (MEG). Proporciona herramientas para leer registros, gestionar canales, eventos y épocas, aplicar técnicas de preprocesado y representar información espacial sobre el cuero cabelludo.
- **NumPy**⁹: librería fundamental de cálculo numérico en Python. Su estructura principal es el array multidimensional, sobre el que se pueden realizar operaciones vectorizadas de forma eficiente.
- **pandas**¹⁰: librería orientada al tratamiento de datos tabulares. Sus estructuras principales, como `DataFrame` y `Series`, permiten organizar, transformar, filtrar y exportar datos de forma cómoda.
- **SciPy**¹¹: conjunto de herramientas científicas construido sobre NumPy. Incluye módulos para estadística, procesamiento de señales, álgebra lineal, optimización y otras operaciones habituales en computación científica.

⁶<https://ubuntu.com/about>

⁷<https://docs.nvidia.com/>

⁸<https://mne.tools/stable/index.html>

⁹<https://numpy.org/doc/stable/>

¹⁰<https://pandas.pydata.org/docs/>

¹¹<https://docs.scipy.org/doc/scipy/>

- **scikit-learn**¹²: librería de aprendizaje automático en Python. Integra modelos de clasificación y regresión, técnicas de validación, métricas, normalización, selección de modelos y métodos de reducción de dimensionalidad.
- **Matplotlib**¹³: librería de visualización que permite crear gráficas y figuras en Python. Se utiliza habitualmente para representar series, distribuciones, matrices, mapas de calor y otros resultados gráficos.
- **joblib**¹⁴: librería pensada para serializar objetos de Python y reutilizarlos posteriormente. Es frecuente en proyectos de aprendizaje automático para guardar modelos, transformaciones o resultados intermedios.

¹²<https://scikit-learn.org/stable/>

¹³<https://matplotlib.org/stable/index.html>

¹⁴<https://joblib.readthedocs.io/en/stable/>

Capítulo 5

Desarrollo del sistema

En este capítulo se describe el desarrollo práctico del sistema implementado para el análisis y clasificación de señales EEG. Se presenta la cadena de procesamiento seguida en el proyecto, desde los datos de partida hasta la obtención de resultados interpretables.

5.1. Flujo general del sistema

La Figura 5.1 resume el flujo seguido por el sistema, indicando en las flechas los datos que pasan de una etapa a la siguiente.

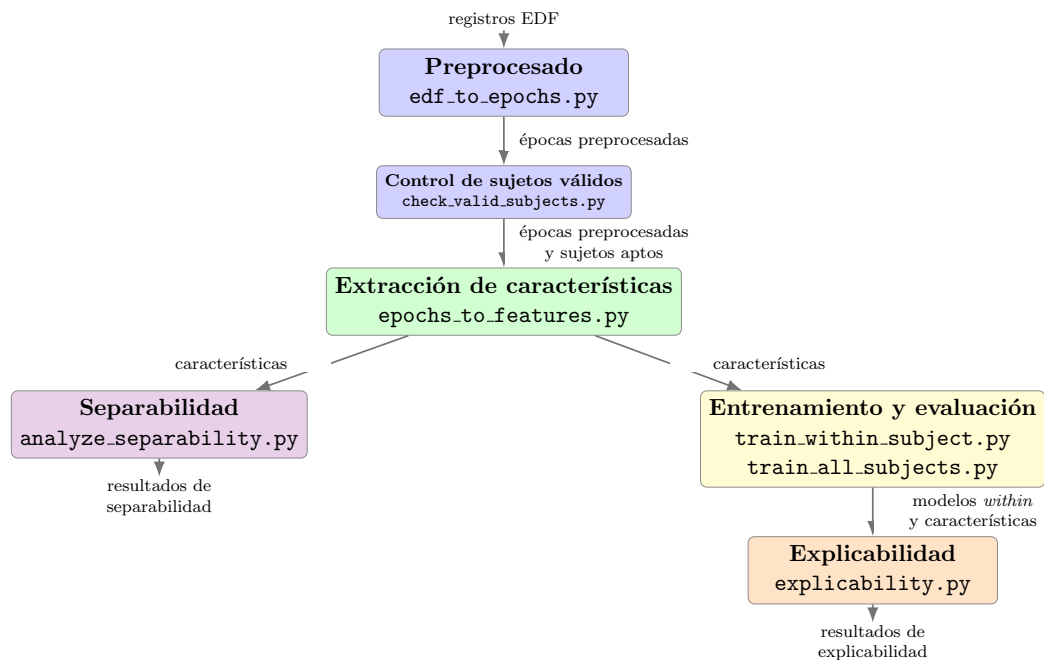


Figura 5.1: Flujo general del sistema desarrollado.

De forma resumida, cada bloque representa una fase concreta del sistema:

- **Preprocesado y control de sujetos válidos.** En esta fase se transforman los registros EDF en épocas de EEG preparadas para el análisis. Además, se revisa qué sujetos tienen datos suficientes y válidos para continuar con las siguientes etapas.
- **Extracción de características.** A partir de las épocas preprocesadas se calculan las variables numéricas que utilizarán los modelos, principalmente características espectrales asociadas a canales y bandas de frecuencia.
- **Análisis de separabilidad.** Este bloque permite estudiar visualmente y mediante métricas si las muestras tienden a agruparse según la emoción o el sujeto antes de entrenar los clasificadores.
- **Entrenamiento y evaluación.** En esta etapa se entrenan los clasificadores interpretables y se evalúa su rendimiento tanto dentro de cada sujeto como considerando varios sujetos conjuntamente.
- **Explicabilidad.** Finalmente, se analizan los modelos obtenidos de cada sujeto para identificar qué características tienen mayor influencia en las predicciones y cómo se distribuye esa importancia sobre canales, bandas y regiones del cuero cabelludo.

5.2. Preprocesado de registros EEG

El primer bloque del sistema corresponde al preprocesado de los registros crudos de EEG. Esta etapa toma como entrada los archivos en formato EDF y genera épocas preparadas para la extracción de características. Una época es un segmento temporal corto extraído de la señal continua alrededor de un evento concreto, de forma que cada fragmento pueda analizarse como una unidad independiente. El objetivo no es modificar la señal de forma arbitraria, sino organizar los datos, reducir fuentes evidentes de ruido y conservar los segmentos asociados a los eventos relevantes de cada grabación.

La implementación se realizó con MNE-Python, una librería orientada al análisis reproducible de señales EEG/MEG que permite trabajar con registros crudos, eventos, filtros, épocas e *Independent Component Analysis* (ICA) dentro de un mismo flujo de procesamiento [25]. Además, se incluyó una fase específica de tratamiento de artefactos, ya que estas señales suelen verse afectadas por interferencias oculares y musculares que pueden alterar el análisis posterior [26].

5.2.1. Carga de archivos y extracción de eventos

Cada archivo EDF se carga inicialmente como un registro continuo, conservando la información completa de la señal y de sus canales auxiliares. A continuación, se extraen los eventos del canal **STIM**, donde se almacenan los marcadores temporales del protocolo. En estos registros pueden aparecer marcas asociadas al comienzo y al final de cada frase, por lo que antes de segmentar la señal es necesario limpiar y seleccionar los eventos que se van a utilizar como referencia.

En primer lugar, se eliminan posibles duplicaciones del mismo marcador. Para ello, si dos eventos aparecen separados por menos de 100 ms, el segundo se descarta. Esta separación es demasiado breve para corresponder a dos frases o respuestas independientes dentro del protocolo, por lo que se interpreta como un rebote del trigger. Con esta limpieza se evita que una misma frase genere dos épocas prácticamente iguales y que el conjunto de datos incluya segmentos repetidos como si fueran ejemplos distintos.

Después de eliminar estos duplicados, el script identifica los eventos de final de frase a partir de la separación temporal entre marcadores consecutivos. Cuando hay dos eventos próximos separados por menos de 2 s, se interpreta que forman un par inicio-fin de frase, y se conserva el segundo evento como referencia para crear la época. En cambio, los intervalos más largos se consideran separaciones entre frases distintas. Si en un registro no aparecen intervalos largos, se asume que el protocolo no guardó un par inicio-fin, sino únicamente los eventos relevantes, por lo que se conservan todos los marcadores disponibles.

Esta selección es importante porque las épocas del análisis se construyen alrededor del final de frase. De esta forma, las etapas posteriores trabajan con segmentos temporales comparables entre archivos, sujetos y condiciones emocionales.

5.2.2. Selección de canales, renombrado y montaje EEG

Tras extraer los eventos de interés, el siguiente paso consiste en preparar la información espacial de los canales. Los registros originales identifican los electrodos mediante una nomenclatura interna basada en códigos alfanuméricos, desde **A1** hasta **H8**. Por ello, primero se seleccionan estos 64 canales principales y se descartan otros canales auxiliares que no forman parte del montaje EEG utilizado para el análisis.

Después, los canales seleccionados se renombran con etiquetas anatómicas reconocibles, como **O1**, **Fz**, **C3** o **Pz**. Este renombrado permite relacionar cada señal con una posición aproximada del cuero cabelludo y facilita las etapas posteriores de análisis espacial, especialmente la generación de mapas topográficos. Dentro de este proceso, los canales **Heor** y **Veol** se marcan como canales oculares, de forma que MNE pueda tratarlos como señales electrooculografía (EOG) durante la detección de artefactos.

Por último, se utilizó el montaje **standard_1005**, una extensión más densa del sistema 10-20¹ que incluye posiciones intermedias y permite representar adecuadamente montajes con mayor número de electrodos. De esta forma, el objeto de señal contiene no solo los valores registrados, sino también la posición espacial asociada a cada electrodo, necesaria para operaciones como la interpolación de canales problemáticos y la representación topográfica de resultados.

5.2.3. Filtrado y remuestreo

Una vez seleccionados y ubicados los canales, se aplican las primeras operaciones de limpieza sobre la señal continua. En primer lugar, se utiliza un filtro *notch*² a 50 Hz para atenuar la interferencia eléctrica de la red³, una fuente de ruido externa a la actividad cerebral y habitual en registros fisiológicos. Después, se aplica un filtro pasa banda entre 0,5 y 40 Hz, manteniendo el rango de frecuencias más relevante para las bandas de EEG analizadas y reduciendo componentes muy lentas o de alta frecuencia que no se emplearán posteriormente.

Tras el filtrado, la señal se remuestrea a 250 Hz cuando la frecuencia original es superior. Remuestrear consiste en cambiar la frecuencia con la que está representada la señal, es decir, el número de muestras por segundo. En este caso se reduce la frecuencia para que los registros trabajen con una resolución temporal común, disminuir el tamaño de los datos y reducir el coste computacional de las siguientes etapas. Si se modifica la frecuencia de muestreo, también se actualiza la posición de los eventos para que sigan apuntando al mismo instante temporal dentro de la señal remuestreada.

¹[https://en.wikipedia.org/wiki/10-20_system_\(EEG\)](https://en.wikipedia.org/wiki/10-20_system_(EEG))

²https://en.wikipedia.org/wiki/Band-stop_filter

³https://en.wikipedia.org/wiki/Utility_frequency

5.2.4. Detección de canales problemáticos y corrección de artefactos

Antes de aplicar la referencia promedio y la corrección de artefactos, el sistema identifica canales cuyo comportamiento se aleja claramente del resto. Esta detección se realiza únicamente sobre los canales EEG, dejando fuera los canales EOG, ya que estos últimos se utilizan como referencia para localizar artefactos oculares.

Para cada canal se calculan varias medidas sobre la señal continua y se comparan con el comportamiento global del resto de canales. La idea no es eliminar canales por tener una amplitud distinta, sino detectar casos claramente anómalos, como electrodos sin contacto, señales casi planas o canales con picos excesivos. Esta lógica multicriterio se planteó tomando como inspiración el procedimiento de detección automática de canales malos descrito por Marta Aguilar Díaz [27], adaptándolo en este trabajo a un conjunto más reducido de métricas temporales.

- **Amplitud pico a pico:** mide la diferencia entre el valor máximo y mínimo de la señal en un canal. Si esta amplitud es extremadamente baja, el canal puede estar prácticamente plano, lo que suele indicar falta de contacto o ausencia de señal útil.
- **Varianza:** resume cuánto cambia la señal a lo largo del tiempo. Una varianza demasiado baja también puede indicar un canal plano, mientras que una varianza muy distinta a la del resto puede señalar ruido o comportamiento inestable.
- **Log-varianza:** se calcula aplicando el logaritmo a la varianza para comparar mejor canales con escalas diferentes. Sobre esta medida se obtiene una puntuación tipo z-score⁴ interna, de forma que los canales muy alejados del conjunto puedan marcarse como anómalos.
- **Curtosis**⁵: describe si la distribución de la señal presenta picos o colas muy pronunciadas. Una curtosis alta puede aparecer cuando un canal contiene artefactos bruscos o valores extremos repetidos.

El z-score se utiliza para expresar cuánto se aleja el valor de un canal respecto al comportamiento medio del conjunto de canales:

⁴https://en.wikipedia.org/wiki/Standard_score

⁵<https://en.wikipedia.org/wiki/Kurtosis>

$$z_i = \frac{x_i - \mu_x}{\sigma_x}$$

Ecuación 5.1: Cálculo del z-score para una métrica de canal.

donde x_i es el valor de la métrica en el canal i , μ_x es la media de esa métrica en todos los canales y σ_x es su desviación típica. De esta forma, un valor alto de $|z_i|$ indica que el canal se aleja mucho del comportamiento habitual del resto.

El criterio final combina estas comprobaciones. Un canal se marca como malo si presenta una señal casi plana, detectada mediante una amplitud pico a pico o una varianza demasiado bajas, o si acumula al menos dos indicadores de comportamiento anómalo. En concreto, se consideran señales de problema la varianza anormal y la curtosis elevada. Esta regla evita descartar un canal por una única medida aislada, pero permite retirar aquellos que muestran varios signos de mala calidad. Los canales detectados se almacenan en la lista de canales malos de MNE, lo que permite que las operaciones posteriores los tengan en cuenta de forma automática.

Después de esta detección se aplica una rereferenciación al promedio. En este paso, cada canal se expresa respecto a la media de los canales EEG disponibles, excluyendo los que ya han sido marcados como malos. Realizar primero la detección de canales problemáticos evita que un canal roto o claramente contaminado influya en la referencia común utilizada para el resto de la señal.

A continuación se aplica ICA, una técnica que descompone la señal en componentes independientes para facilitar la separación de actividad cerebral y artefactos. En este trabajo se ajustan 20 componentes mediante el método *FastICA*⁶. Los canales *Heor* y *Veol*, marcados previamente como EOG, se utilizan para detectar componentes relacionados con parpadeos y movimientos oculares. Además, el sistema intenta identificar componentes con patrón muscular, asociados a actividad EMG. Para evitar eliminar demasiada información útil, el número máximo de componentes excluidos se limita a cinco.

Finalmente, una vez aplicada la corrección mediante ICA, los canales marcados como malos se interpolan a partir de la información de los canales vecinos. La interpolación no recupera la señal original exacta del electrodo defectuoso, sino que estima una señal coherente con la distribución espacial del resto de canales y con el

⁶<https://en.wikipedia.org/wiki/FastICA>

montaje asignado. De esta forma, el conjunto final mantiene una estructura completa de canales, necesaria para las etapas posteriores de extracción de características y representación espacial.

5.2.5. Segmentación en épocas y rechazo de segmentos anómalos

Una vez corregida la señal continua, se crean las épocas que se utilizarán en las etapas posteriores. Cada época se construye tomando como referencia un evento de final de frase, seleccionado previamente a partir del canal **STIM**. La ventana temporal utilizada va desde 1,5 s antes del evento hasta 1,0 s después, por lo que cada segmento incluye la parte final del estímulo auditivo y la actividad inmediatamente posterior.

En esta etapa no se aplica corrección de línea base, ya que la normalización respecto a la condición neutra se realiza más adelante durante la extracción de características. De esta forma, las épocas se guardan como fragmentos temporales limpios, pero todavía sin transformar en variables espectrales.

Tras crear las épocas, se aplica un rechazo automático de segmentos con amplitud anómala. Para cada época se calcula la amplitud pico a pico de cada canal, y se marca como problemática si más de tres canales superan un umbral de $80\mu V$. Este criterio evita descartar una época por un pico aislado en un único canal, pero permite eliminar segmentos claramente contaminados por movimientos, artefactos musculares u otras alteraciones de gran amplitud.

Si después de este rechazo un archivo queda sin épocas válidas, el procesamiento de ese registro se detiene y se informa del error. Esto evita que archivos sin segmentos útiles pasen a las fases de extracción de características y entrenamiento.

5.2.6. Almacenamiento de épocas y estadísticas de control

El último paso del preprocesado consiste en guardar los resultados generados para cada registro. Las épocas válidas se almacenan en formato *Functional Imaging File* (FIF), el formato utilizado por MNE para conservar objetos de señal ya procesados. Los archivos se organizan según la condición emocional correspondiente, de forma que las etapas posteriores puedan localizar fácilmente las épocas asociadas a alegría, estado neutro o tristeza.

Además del archivo de épocas, se genera un fichero *JavaScript Object Notation* (JSON) con estadísticas de control del preprocesado. Este resumen incluye el nombre del registro, la emoción asociada, el número final de épocas conservadas, las épocas rechazadas, los canales marcados como malos, los componentes eliminados mediante ICA y la frecuencia de muestreo final. Estos datos permiten revisar qué ocurrió en cada archivo y facilitan el control de calidad antes de pasar a la extracción de características.

La salida de esta etapa queda formada por dos elementos principales: por un lado, las épocas preprocesadas que alimentan el siguiente bloque del sistema; por otro, las estadísticas que permiten auditar el proceso y detectar registros con posibles problemas.

5.2.7. Resumen

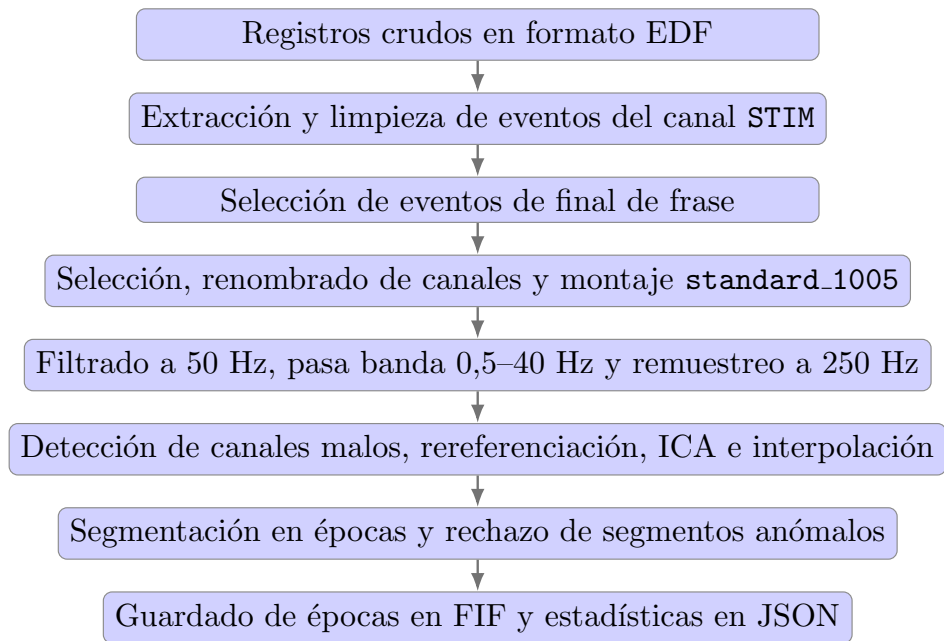


Figura 5.2: Resumen del flujo de preprocesado aplicado a cada registro EEG.

5.2.8. Control de sujetos válidos

Después del preprocesado se realiza una comprobación auxiliar para identificar qué sujetos pueden continuar en el flujo del sistema. Este paso no modifica las señales, sino que revisa las épocas generadas y evita que sujetos incompletos o con señales claramente anómalas pasen a la extracción de características.

El control aplica tres criterios principales:

- **Completitud por sujeto:** se comprueba que cada participante tenga épocas preprocesadas para las tres condiciones emocionales del trabajo: alegría, estado neutro y tristeza. También se revisan los archivos crudos disponibles, lo que permite distinguir entre una condición no grabada y una condición que existía en crudo pero no llegó a generar épocas válidas.
- **Balance de épocas:** para los sujetos completos se cuenta el número de épocas válidas en cada emoción. Si la condición con menos épocas queda por debajo del 50 % de la condición con más épocas, el sujeto se marca con un aviso de desequilibrio. Este criterio no descarta automáticamente al sujeto, pero permite revisar posibles diferencias importantes entre condiciones.
- **Revisión espectral automática:** se calcula el espectro medio de cada sujeto y condición entre 1 y 40 Hz. El control exige que exista un pico alfa suficiente frente a la banda beta, usando una relación alfa/beta mínima de 1,30. También se comprueba que la potencia al inicio del espectro analizado, alrededor de 1 Hz, sea al menos tres veces mayor que la potencia al final, alrededor de 40 Hz. Esta relación se calcula como P_{1Hz}/P_{40Hz} y permite detectar señales con un espectro demasiado plano o poco compatible con la forma habitual de una señal EEG. Si un sujeto no supera esta revisión espectral, se excluye de las etapas posteriores.

Como salida, el sistema genera una lista de sujetos aptos, una lista de sujetos descartados con el motivo correspondiente, un resumen en formato JSON y figuras de apoyo para revisar el balance de épocas y los espectros individuales. De esta forma, las siguientes etapas trabajan con participantes que mantienen información comparable entre condiciones y una calidad mínima de señal.

5.3. Extracción de características

Una vez obtenidas las épocas preprocesadas, el siguiente bloque transforma cada segmento de EEG en una representación numérica adecuada para los análisis posteriores. En lugar de utilizar directamente la señal temporal completa, se extraen variables espectrales, es decir, medidas que describen cómo se reparte la actividad de la señal entre distintas frecuencias. Una característica puede indicar, por ejemplo, cuánta potencia presenta un canal concreto en la banda Alpha durante una época determinada.

Este enfoque permite resumir cada segmento mediante valores comparables entre

sujetos y condiciones. Para ello se calcula la información espectral de cada época, se agrupa por bandas de frecuencia y se normaliza respecto a la condición neutra de cada sujeto. El resultado final es un conjunto de matrices y metadatos que permite entrenar los modelos de clasificación y relacionar cada muestra con su sujeto, emoción y época original.

5.3.1. Carga de épocas y sujetos válidos

La extracción de características parte de la lista de sujetos aptos generada en el control anterior. El script lee el archivo `valid_subjects.txt` y, para cada sujeto, busca las épocas preprocesadas correspondientes a las condiciones JOY, NEUTRO y SAD. Cada registro se carga desde su archivo FIF, manteniendo la correspondencia entre sujeto, emoción y número de época.

Una vez cargado cada archivo, se seleccionan únicamente los canales EEG. Con ello se evita que canales auxiliares, como los EOG, entren en el cálculo de características espectrales. Esta decisión mantiene la matriz final centrada en la actividad cerebral registrada y deja las señales auxiliares solo como apoyo del preprocesado.

5.3.2. Cálculo de potencia espectral por bandas

Para convertir cada época en características espectrales se necesita pasar de la señal en el tiempo a una representación en frecuencia. En el dominio temporal se observa cómo cambia la amplitud de la señal a lo largo de los segundos; en el dominio frecuencial se observa qué frecuencias están más presentes en esa señal. Para realizar esta transformación se utiliza la *Fast Fourier Transform* (FFT)⁷, que calcula de forma eficiente la transformada discreta de Fourier:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N}$$

Ecuación 5.2: Transformada discreta de Fourier calculada de forma eficiente mediante *FFT*.

donde x_n representa la señal en el tiempo, N es el número de muestras analizadas y X_k representa la contribución de una frecuencia concreta. Cada época dura 2,5 s y la

⁷https://en.wikipedia.org/wiki/Fast_Fourier_transform

señal está remuestreada a 250 Hz, por lo que cada segmento contiene $2,5 \cdot 250 = 625$ muestras. Por tanto, el análisis frecuencial se realiza sobre esos 625 puntos de señal.

La Figura 5.3 muestra un ejemplo sencillo de una señal formada por componentes de 2 Hz, 10 Hz y 20 Hz. En el dominio temporal estas componentes aparecen mezcladas, mientras que en el dominio frecuencial se observan como picos en las frecuencias correspondientes. Esta idea permite entender el cálculo de la *Power Spectral Density* (PSD), que estima cómo se distribuye la potencia de una señal a lo largo de la frecuencia.

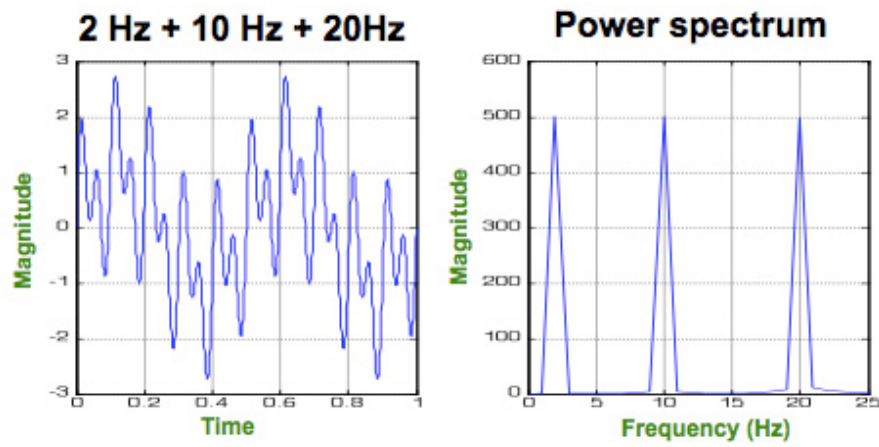


Figura 5.3: Ejemplo de señal en dominio temporal y frecuencial. Fuente: [28].

En lugar de calcular el espectro una única vez sobre toda la señal, el script estima la PSD mediante el método de Welch [29]. Este método divide la señal en ventanas, calcula el espectro de cada una mediante la FFT y después promedia los resultados. Con ello se obtiene una estimación más estable de la potencia por frecuencia que la que se obtendría con un único cálculo directo.

$$\widehat{\text{PSD}}_{\text{Welch}}(f) = \frac{1}{K} \sum_{k=1}^K P_k(f)$$

Ecuación 5.3: Estimación de la densidad espectral de potencia mediante el método de Welch.

donde $P_k(f)$ es el periodograma de la ventana k , K es el número de ventanas analizadas y f representa la frecuencia. Además, se utiliza un solapamiento del 50 %, lo que significa que una ventana comparte la mitad de sus muestras con la siguiente. Este

solapamiento permite aprovechar mejor la información temporal disponible y suavizar la estimación final de la PSD. La estimación se limita al rango entre 1 y 40 Hz, que contiene las bandas de EEG utilizadas posteriormente.

Después, la PSD no se utiliza frecuencia a frecuencia, porque eso generaría un número elevado de variables y sería menos interpretable. En su lugar, se resume en bandas conocidas de actividad EEG: Delta (1 – 4 Hz), Theta (4 – 8 Hz), Alpha (8 – 12 Hz), Beta (12 – 30 Hz) y Gamma (30 – 40 Hz). Para cada época, canal y banda, se calcula la potencia media de las frecuencias incluidas en ese intervalo:

$$P_{e,c,b} = \frac{1}{|F_b|} \sum_{f \in F_b} \text{PSD}_{e,c}(f)$$

Ecuación 5.4: Potencia media de una banda de frecuencia para una época y un canal.

donde $P_{e,c,b}$ es la potencia media de la época e , el canal c y la banda b ; F_b representa el conjunto de frecuencias pertenecientes a esa banda. Así, cada época queda descrita por valores del tipo “canal-banda”, como la potencia Alpha en F3 o la potencia Beta en Pz. El resultado de este paso mantiene una estructura tridimensional, con dimensiones época, canal y banda, que sirve como base para las transformaciones posteriores.

5.3.3. Normalización respecto a la condición neutra

Las potencias obtenidas mediante la PSD pueden tener escalas muy distintas entre canales, bandas y sujetos. Además, la potencia espectral suele variar varios órdenes de magnitud, por lo que comparar los valores directamente puede hacer que las bandas o canales con mayor amplitud dominen el análisis. Para reducir este efecto, el script transforma primero la potencia a escala logarítmica mediante $\log_{10}$. En la implementación se añade un valor $\epsilon = 10^{-30}$ antes del logaritmo para evitar problemas numéricos si apareciera una potencia nula.

Tabla 5.1: Ejemplo de transformación logarítmica en base 10.

Valor original x	$\log_{10}(x)$	Interpretación
1000	3	Valor alto en escala lineal.
10	1	Valor menor, pero todavía por encima de la unidad.
1	0	Punto de referencia de la escala logarítmica.
0,001	-3	Valor pequeño, expresado como potencia de 10.
0,000001	-6	Valor muy pequeño, pero más fácil de comparar en esta escala.

$$L_{e,c,b} = \log_{10}(P_{e,c,b} + \epsilon)$$

Ecuación 5.5: Transformación logarítmica de la potencia espectral.

donde $P_{e,c,b}$ es la potencia media de la época e , el canal c y la banda b , y $L_{e,c,b}$ es su valor en escala logarítmica. Esta transformación no cambia qué épocas tienen más o menos potencia, pero comprime la escala y facilita la comparación entre variables.

Después se calcula una referencia propia para cada sujeto usando sus épocas de la condición **NEUTRO**. Esta condición se utiliza como línea base porque representa el estado no emocional frente al que se quieren comparar las condiciones de alegría y tristeza. La referencia se obtiene promediando las potencias logarítmicas neutras de cada canal y banda:

$$B_{s,c,b} = \frac{1}{|E_{s,\text{NEUTRO}}|} \sum_{e \in E_{s,\text{NEUTRO}}} L_{e,c,b}$$

Ecuación 5.6: Referencia neutra de un sujeto para cada canal y banda.

donde $B_{s,c,b}$ es la referencia del sujeto s , y $E_{s,\text{NEUTRO}}$ es el conjunto de épocas neutras de ese sujeto. A partir de esta referencia se calcula el cambio logarítmico de cada época respecto al estado neutro:

$$\Delta_{e,c,b} = L_{e,c,b} - B_{s,c,b}$$

Ecuación 5.7: Diferencia logarítmica respecto a la condición neutra del mismo sujeto.

De esta forma, las características no representan únicamente la potencia absoluta de una época, sino cuánto se aleja esa potencia del patrón neutro del mismo participante. Esto es importante porque dos sujetos pueden tener niveles de potencia diferentes por motivos ajenos a la emoción, como diferencias fisiológicas, colocación de electrodos o calidad de contacto.

Por último, los bloques de características generados a partir de estos valores se normalizan mediante z-score por sujeto, usando de nuevo la condición **NEUTRO** como referencia. Para cada característica se calcula la media y la desviación típica de las épocas neutras del sujeto, y el resto de condiciones se expresan en esa misma escala:

$$Z_{e,j}^{(s)} = \frac{X_{e,j}^{(s)} - \mu_{j,\text{NEUTRO}}^{(s)}}{\sigma_{j,\text{NEUTRO}}^{(s)}}$$

Ecuación 5.8: Normalización z-score por sujeto usando la condición neutra como referencia.

donde $X_{e,j}^{(s)}$ es el valor de la característica j en una época del sujeto s , $\mu_{j,\text{NEUTRO}}^{(s)}$ y $\sigma_{j,\text{NEUTRO}}^{(s)}$ son la media y desviación típica de esa característica en las épocas neutras del mismo sujeto. Con esta normalización, los modelos reciben variables más comparables entre participantes y centradas en los cambios relativos respecto a su propio estado neutro.

En resumen, la resta respecto a **NEUTRO** elimina el nivel base propio de cada participante y convierte las variables en cambios respecto a su estado neutro. El z-score añade una segunda corrección: expresa esos cambios según la variabilidad normal de cada sujeto, de forma que una misma diferencia no tenga el mismo peso si en un participante es habitual y en otro es excepcional.

5.3.4. Construcción del conjunto de características

Tras calcular los cambios respecto a la condición neutra, cada época todavía conserva una estructura tridimensional: época, canal y banda. Para poder trabajar con estos valores como una tabla de datos, el script los transforma en una matriz bidimensional, donde cada fila representa una época y cada columna una característica concreta. Esta conversión se realiza agrupando las variables en tres bloques interpretables: potencia por banda, relación Theta/Alpha y asimetría Alpha.

El primer bloque corresponde a la potencia por canal y banda. Para cada época se toma la matriz formada por los 62 canales EEG y las cinco bandas espectrales, y se aplanan en una única fila. De este modo, se obtienen $62 \cdot 5 = 310$ variables con nombres del tipo `FC4_Delta`, `FC4_Theta`, `FC4_Alpha`, `FC4_Beta` o `FC4_Gamma`. Cada una indica el cambio de potencia de una banda concreta en un canal concreto respecto al patrón neutro del sujeto. El uso de potencias espectrales por bandas es habitual en reconocimiento de emociones con EEG, y bases de datos como DEAP emplean características de potencia por bandas y diferencias entre electrodos simétricos [2, 30].

El segundo bloque resume la relación entre Theta y Alpha para cada canal. Se genera una característica por canal, por lo que este bloque añade 62 variables nuevas con nombres como `ThetaAlphaRatio_FC4`. Como los valores ya están en escala logarítmica y expresados respecto a `NEUTRO`, esta relación se calcula como una diferencia entre ambas bandas:

$$R_{e,c}^{\Theta/\text{Alpha}} = \Delta_{e,c,\Theta} - \Delta_{e,c,\text{Alpha}}$$

Ecuación 5.9: Relación Theta/Alpha calculada como diferencia en escala logarítmica.

donde $R_{e,c}^{\Theta/\text{Alpha}}$ representa la relación Theta/Alpha de la época e en el canal c . Al trabajar en escala logarítmica, restar Alpha a Theta equivale a comparar ambas potencias de forma relativa. Así, la característica no mide solo si hay mucha Theta o mucha Alpha por separado, sino si una de las dos aumenta más que la otra respecto al estado neutro. Este tipo de relación entre bandas se ha utilizado como característica derivada en análisis de EEG, especialmente para comparar la actividad relativa entre Alpha y Theta [31].

El tercer bloque recoge la asimetría Alpha entre pares de electrodos homólogos de los hemisferios izquierdo y derecho. La asimetría en banda Alpha, especialmente en zonas frontales, se ha utilizado en estudios sobre emoción y motivación, y también aparece en trabajos de análisis emocional con EEG mediante diferencias entre pares simétricos de electrodos [30, 32]. Para cada par disponible se calcula la diferencia entre el canal derecho y el canal izquierdo. En el conjunto utilizado hay 26 pares, por lo que se añaden 26 variables de asimetría con nombres como `AlphaAsym_FC4-FC3`, `AlphaAsym_F4-F3` o `AlphaAsym_CP4-CP3`:

$$A_{e,(l,r)} = \Delta_{e,r,\text{Alpha}} - \Delta_{e,l,\text{Alpha}}$$

Ecuación 5.10: Asimetría Alpha entre un par de canales homólogos.

donde l y r representan, respectivamente, el canal izquierdo y derecho del par. Esta variable permite resumir diferencias laterales de la banda Alpha sin tratar cada canal de forma aislada.

Una vez generados los bloques, cada uno se normaliza de forma independiente mediante el procedimiento descrito en la sección anterior. Esta separación evita mezclar distribuciones distintas, ya que las potencias por banda, las relaciones Theta/Alpha y las asimetrías Alpha no tienen exactamente la misma naturaleza. Finalmente, los bloques normalizados se concatenan por columnas para formar la fila definitiva de cada época. En total, el conjunto completo generado por la extracción queda formado por $310 + 62 + 26 = 398$ características por época.

Cada fila del conjunto de características queda asociada a una etiqueta numérica: JOY se codifica como 0, NEUTRO como 1 y SAD como 2. Por tanto, cada sujeto aporta tantas filas como épocas válidas conserve tras el preprocesado, sumando las tres condiciones emocionales disponibles. Además, durante el ensamblado se mantiene la información necesaria para saber de qué sujeto, condición e índice de época procede cada muestra. El primer sujeto procesado fija los nombres y el orden de las columnas; si otro sujeto generase una estructura distinta, el script detendría la ejecución para evitar matrices incompatibles.

Tabla 5.2: Ejemplo simplificado de la estructura del conjunto de características.

Sujeto	Condición	Época	Etiqueta	F3.Alpha	Pz.Beta	ThetaAlphaRatio.F3	AlphaAsym.F4-F3	...
211-000531	JOY	0	0	0.84	-0.21	1.12	-0.35	...
211-000531	NEUTRO	0	1	-0.06	0.18	-0.14	0.09	...
211-000531	SAD	0	2	-0.47	0.62	-0.73	0.41	...
211-000532	JOY	0	0	0.31	-0.58	0.44	-0.12	...

Los valores numéricos de la Tabla 5.2 son ilustrativos. La tabla muestra la idea general: las primeras columnas identifican la muestra y su etiqueta, mientras que el resto corresponden a características calculadas a partir de la señal. La matriz de características contiene únicamente las columnas numéricas generadas a partir del EEG; la información de sujeto, condición e índice de época se conserva como metadatos para poder interpretar y revisar los resultados.

5.3.5. Guardado y check

El último paso de la extracción consiste en guardar las matrices y los metadatos necesarios para las fases posteriores. Esta separación permite conservar, por un lado, los valores numéricos que entran a los modelos y, por otro, la información que permite interpretar de dónde procede cada fila.

Los archivos generados se pueden dividir en dos grupos:

Archivos utilizados en entrenamiento y evaluación

- **features_X.npy**: contiene la $\log_{10}(\text{PSD})$ por canal y banda, sin aplicar todavía el delta respecto a NEUTRO ni la normalización final. Es la matriz que cargan los scripts de entrenamiento como punto de partida. A partir de ella, cada partición de validación calcula su propia referencia neutra y su propia normalización usando solo los datos de entrenamiento, evitando que las muestras de prueba influyan en el ajuste.
- **features_y.npy**: almacena la etiqueta numérica asociada a cada época, es decir, la emoción correspondiente a cada fila de la matriz de características.

Archivos de comprobación e información

- **features_meta.csv**: conserva información descriptiva de cada muestra, como el sujeto, la condición y el índice de época. No se utiliza como característica de entrada del modelo, pero sí permite organizar la validación, separar sujetos y relacionar cada predicción con su origen.
- **features_X_norm.npy**: contiene el conjunto completo ya transformado, con $\log_{10}$, delta respecto a NEUTRO, bloques derivados y z-score por sujeto. Esta versión es útil para revisar la estructura final de las características y realizar comprobaciones generales, pero no es la entrada principal de los entrenamientos con validación cruzada.
- **features_info.json**: guarda la configuración de la extracción: bandas utilizadas, canales, parámetros de Welch, nombres de características, número de sujetos, distribución de clases y parámetros de normalización.

Además del guardado, el script realiza un diagnóstico final del conjunto generado. Esta comprobación no crea nuevas características ni modifica las existentes, sino que sirve para detectar errores antes de pasar al entrenamiento. En concreto, se revisan las dimensiones de las matrices, la distribución de épocas por clase, el número de épocas aportadas por cada sujeto y estadísticas básicas de las características por condición. También se comprueba que no existan valores **NaN** ni infinitos, ya que estos impedirían entrenar correctamente los modelos. Por último, se calcula la media de las características asociadas a **NEUTRO**. Como la normalización se ha construido usando esta condición como referencia, esa media debería quedar cercana a cero. Si se alejara demasiado, sería una señal de que habría que revisar el cálculo del delta o del z-score.

5.3.6. Resumen

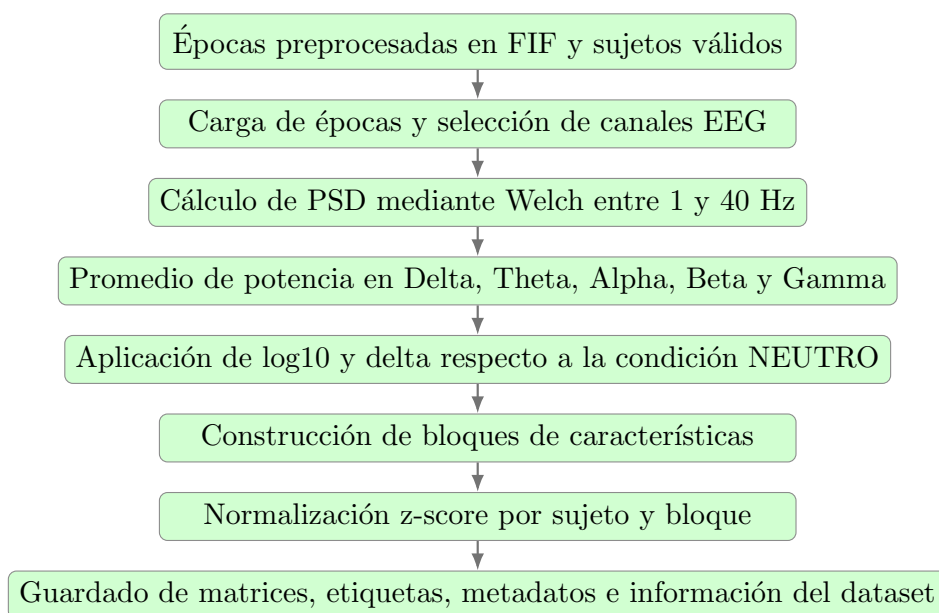


Figura 5.4: Resumen del flujo de extracción de características aplicado a las épocas preprocesadas.

5.4. Entrenamiento y evaluación de modelos

Una vez generadas las características, el siguiente bloque del sistema entrena y evalúa los clasificadores. Para ello se implementaron dos programas: `train_within_subject.py`, orientado a modelos individuales por sujeto, y `train_all_subjects.py`, destinado a entrenar un modelo global con varios participantes. Ambos parten de los mismos archivos de características, pero aplican estrategias de validación diferentes.

La normalización respecto a la condición **NEUTRO** se calcula dentro del entrenamiento, después de separar los datos de cada partición. Así se evita que las muestras de prueba participen en el cálculo de referencias o escalados.

5.4.1. Entrada al entrenamiento

Los scripts cargan **features_X.npy** como matriz base de características, **features_y.npy** como vector de etiquetas y **features_meta.csv** para conocer el sujeto, la condición y el índice de época de cada fila. Además, leen **features_info.json** para recuperar la lista de canales y comprobar que la forma de la matriz coincide con la estructura esperada.

La entrada principal es **features_X.npy**, no **features_X_norm.npy**. La versión normalizada se reserva para comprobaciones, ya que en los entrenamientos la referencia neutra y el escalado se recalculan dentro de cada partición usando solo los datos de entrenamiento.

5.4.2. Construcción de características dentro de cada partición

Aunque la extracción de características genera una versión completa de 398 variables, los scripts de entrenamiento construyen una matriz más compacta a partir de **features_X.npy**. Esta matriz base contiene cinco bandas por canal, pero el entrenamiento selecciona únicamente las bandas Alpha y Beta y añade las asimetrías Alpha entre pares hemisféricos. Con ello se reduce el número de variables y se trabaja con un conjunto más fácil de interpretar.

La selección de Alpha y Beta responde al contexto del registro y a un criterio conservador. En una tarea de escucha pasiva, con participantes en reposo y en un entorno controlado, estas bandas son especialmente adecuadas para resumir cambios asociados al estado atencional y al procesamiento activo de los estímulos auditivos. En cambio, Delta y Theta se dejaron fuera de la configuración principal por estar más relacionadas con actividad lenta, somnolencia o baja activación, lo que podía aportar información menos específica para esta tarea.

La banda Gamma tampoco se incluyó en el entrenamiento principal. Esto no significa que Gamma carezca de interés en cualquier análisis de EEG, sino que en

este caso se adoptó una interpretación prudente: al tratarse de frecuencias más altas, pueden verse más afectadas por actividad muscular, contacto de electrodos, pelo u otros artefactos residuales. Si un modelo dependiera mucho de esta banda, la interpretación sería menos segura, porque parte de esa información podría proceder de ruido no cerebral. Por este motivo se priorizó un conjunto más estable formado por Alpha, Beta y asimetría Alpha.

El proceso se realiza dentro de cada partición de validación. Primero, la matriz de entrenamiento se reorganiza internamente para recuperar la estructura época-canal-banda. Después se calcula una referencia de la condición **NEUTRO** usando solo las muestras de entrenamiento de esa partición. Esa referencia se resta tanto a las muestras de entrenamiento como a las de prueba, de forma que cada valor representa un cambio respecto al estado neutro estimado sin utilizar información del conjunto de prueba.

Tras calcular esta diferencia respecto a **NEUTRO**, se ajusta el escalado con los datos de entrenamiento y se aplica la misma transformación al conjunto de prueba. Este orden es importante: si el escalado se calculase usando todas las muestras, el modelo recibiría información indirecta sobre los datos que debería predecir después.

La matriz final que recibe el clasificador se compone de tres bloques:

- **Potencia Alpha por canal:** una variable por cada canal EEG.
- **Potencia Beta por canal:** una variable por cada canal EEG.
- **Asimetría Alpha:** diferencias entre pares de electrodos homólogos izquierdo-derecho.

Con 62 canales, las dos primeras partes aportan $62 \cdot 2 = 124$ características. A ellas se suman 26 variables de asimetría Alpha, por lo que cada época queda representada finalmente por 150 características. Esta construcción se aplica de la misma forma en los dos programas de entrenamiento; lo que cambia entre ellos es la forma de dividir los datos para evaluar los modelos.

5.4.3. Clasificadores utilizados

Los dos programas de entrenamiento utilizan los mismos tipos de clasificador: regresión logística y árbol de decisión. La elección mantiene la línea interpretativa

planteada en los fundamentos teóricos, ya que ambos modelos permiten revisar qué variables intervienen en la decisión sin depender únicamente de una predicción final.

La regresión logística se utiliza como modelo lineal principal. En los scripts se entrena con regularización *elastic-net*⁸, lo que ayuda a controlar el peso de las características y reduce el riesgo de ajustar demasiado el modelo a particularidades del conjunto de entrenamiento. Además, al disponer de coeficientes asociados a las variables de entrada, sus resultados pueden relacionarse después con canales, bandas y asimetrías concretas. El árbol de decisión se utiliza como segundo modelo interpretable. Su papel dentro del sistema es servir como comparación frente al modelo lineal, ya que puede capturar divisiones no lineales entre características.

Ambos clasificadores se entrenan con ponderación balanceada de clases. Esta opción ajusta el peso de cada clase durante el aprendizaje para compensar posibles diferencias en el número de épocas disponibles de JOY, NEUTRO y SAD. De este modo, una clase con menos muestras no queda tan fácilmente ignorada por el modelo.

La diferencia entre los dos scripts no está en los clasificadores utilizados, sino en la forma de evaluarlos. En `train_within_subject.py` se emplea una configuración fija para cada modelo, mientras que en `train_all_subjects.py` se realiza una búsqueda de hiperparámetros⁹ probando combinaciones predefinidas antes de guardar el modelo global final.

5.4.4. Entrenamiento *within-subject*

La estrategia *within-subject* evalúa el comportamiento de los modelos dentro de cada participante de forma independiente. Para ello, el script recorre los sujetos disponibles y, en cada caso, selecciona únicamente sus épocas de JOY, NEUTRO y SAD. Así, el modelo de un sujeto se entrena y se evalúa solo con datos de ese mismo sujeto.

Antes de entrenar, se comprueba que el participante tenga muestras suficientes de las tres clases. Si alguna emoción tiene menos épocas que el número de particiones necesarias, ese sujeto se descarta para evitar una validación incompleta. En los sujetos válidos se construyen cinco particiones temporales. Dentro de cada emoción, las épocas se ordenan mediante `epoch_idx` y se dividen en cinco tramos contiguos, sin mezclar aleatoriamente las muestras.

⁸https://en.wikipedia.org/wiki/Elastic_net_regularization

⁹https://en.wikipedia.org/wiki/Hyperparameter_optimization

En cada fold se reserva como prueba un tramo temporal de cada emoción y se entrena con los tramos restantes. Esta organización mantiene las tres clases representadas en cada evaluación y evita que el conjunto de prueba se forme con épocas elegidas al azar de toda la sesión. Después de separar entrenamiento y prueba, se calcula la referencia **NEUTRO**, se aplica el escalado correspondiente y se construyen las características Alpha, Beta y asimetría Alpha descritas en la sección anterior.

Los clasificadores se entrenan con una configuración fija, sin búsqueda de hiperparámetros. Esta decisión simplifica la comparación entre sujetos y evita añadir una segunda capa de selección dentro de conjuntos de datos relativamente pequeños. Para cada participante y clasificador se calculan las métricas de los cinco folds, se acumulan las predicciones y se guarda un resumen con la media y desviación típica del rendimiento.

Una vez terminada la evaluación, el script entrena modelos finales por sujeto usando todas sus épocas válidas. Estos modelos se guardan junto con la información de preprocesado necesaria para aplicar después las mismas transformaciones a nuevos datos del mismo participante.

5.4.5. Entrenamiento *all-subjects*

La estrategia *all-subjects* entrena un modelo global a partir de los datos de todos los participantes válidos. A diferencia del caso anterior, el objetivo no es ajustar un modelo específico para cada persona, sino comprobar si existe un patrón común que permita clasificar emociones en sujetos no usados durante el entrenamiento.

Para evaluar este escenario se utiliza **StratifiedGroupKFold**¹⁰, tomando el identificador del sujeto como grupo. De esta forma, las épocas de un mismo participante no pueden aparecer simultáneamente en entrenamiento y prueba. Esta separación es importante porque varias épocas del mismo sujeto pueden compartir patrones individuales que no deberían filtrarse al conjunto de prueba. Además, la parte estratificada de la partición ayuda a mantener una distribución de clases lo más equilibrada posible entre folds.

En esta estrategia sí se realiza búsqueda de hiperparámetros. Para cada clasificador se define una rejilla de combinaciones mediante **ParameterGrid**¹¹. Cada combinación se

¹⁰Documentación de **StratifiedGroupKFold** en scikit-learn

¹¹Documentación de **ParameterGrid** en scikit-learn

evalúa con la misma validación por grupos. En cada fold, igual que en el entrenamiento por sujeto, la referencia **NEUTRO** y el escalado se calculan solo con los datos de entrenamiento, y después se aplican al conjunto de prueba.

El criterio principal para seleccionar la mejor configuración es el F1 macro, ya que resume el rendimiento medio entre clases sin favorecer únicamente a la clase más representada. Si dos configuraciones obtienen un valor similar, la exactitud se usa como criterio secundario. Finalmente, el script entrena un modelo global final por clasificador usando todos los datos disponibles y los mejores hiperparámetros encontrados.

5.4.6. Métricas, figuras y modelos guardados

Al finalizar cada evaluación, los scripts almacenan tanto los resultados numéricos como los objetos necesarios para revisar o reutilizar los modelos. Las métricas calculadas son la exactitud y el F1 macro, guardando siempre la media y la desviación típica cuando la evaluación se realiza por folds. Además, se acumulan las etiquetas reales y predichas para construir matrices de confusión agregadas.

En el entrenamiento *within-subject*, los resultados se organizan por sujeto y clasificador. El archivo `within_subject_summary.csv` contiene una tabla resumida con las métricas de cada participante, mientras que `within_subject_results.json` conserva la misma información en formato estructurado, incluyendo el clasificador seleccionado como mejor opción global. También se guarda `selected_features.csv`, con la lista de características utilizadas finalmente por los modelos.

Para cada clasificador *within-subject* se genera una figura general de exactitud: cada barra corresponde a un sujeto y la línea de media resume el rendimiento conjunto de todos los participantes evaluados. Además, se genera una matriz de confusión agregada, construida acumulando las predicciones de todos los sujetos para ese clasificador. Junto a la matriz se guarda un *classification report* en formato JSON, que resume el comportamiento por clase. Por último, se entrena un modelo final para cada sujeto válido y cada clasificador, almacenándolo en formato `joblib` dentro de las subcarpetas `logReg` y `decisionTree`.

En el entrenamiento *all-subjects*, el archivo principal es `all_subjects_results.json`. Este fichero guarda, para cada clasificador, la mejor combinación de hiperparámetros, las métricas por fold, los sujetos que han formado cada partición de prueba y los resultados de todas las combinaciones evaluadas durante la búsqueda.

De esta forma, no solo queda registrado el modelo ganador, sino también el proceso seguido para seleccionarlo.

Las figuras del caso *all-subjects* son más completas porque existe un único modelo global por clasificador. Para cada uno se genera una gráfica con la exactitud y el F1 macro por fold, una matriz de confusión absoluta y normalizada, y una gráfica de importancia de características. Además, después de entrenar los modelos finales con todos los datos disponibles, se generan visualizaciones específicas: coeficientes principales para la regresión logística y una representación de los primeros niveles del árbol de decisión.

Los modelos finales también se guardan en formato `joblib`. Cada archivo incluye el clasificador entrenado, los parámetros utilizados, los nombres de canales y características, el mapa de etiquetas y la información de preprocesado necesaria para aplicar la misma transformación a nuevos datos. Esto evita que el modelo quede separado de las operaciones previas que necesita para recibir las características en el mismo formato con el que fue entrenado.

5.4.7. Resumen

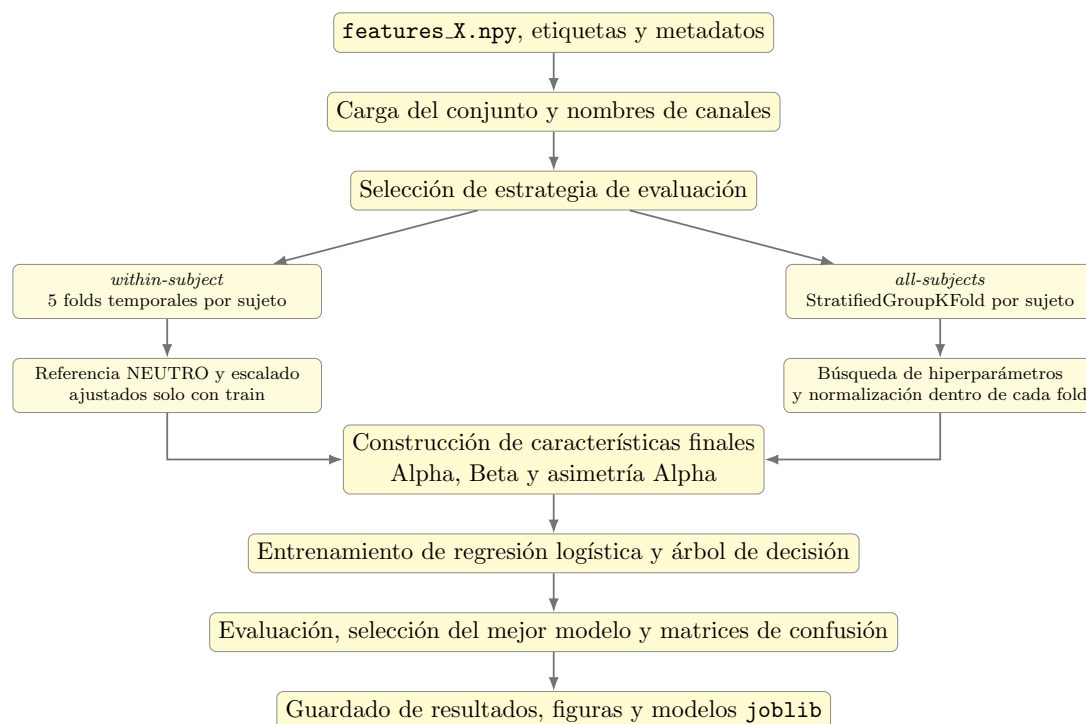


Figura 5.5: Resumen del flujo de entrenamiento y evaluación de modelos.

5.5. Análisis de separabilidad

Además del entrenamiento de clasificadores, se añadió un análisis de separabilidad para revisar cómo se distribuyen las muestras en el espacio de características. Este bloque no entrena un nuevo modelo predictivo, sino que permite observar si las características utilizadas tienden a agrupar las épocas por emoción o si, por el contrario, aparecen agrupaciones más relacionadas con el sujeto.

El script `analyze_separability.py` utiliza el mismo conjunto de variables que el entrenamiento principal y genera representaciones bidimensionales mediante *Principal Component Analysis* (PCA)¹² y *Linear Discriminant Analysis* (LDA)¹³. Para interpretar estas figuras conviene distinguir el papel de ambas proyecciones. PCA no utiliza las etiquetas de emoción, sino que busca las direcciones donde los datos varían más. LDA, en cambio, sí utiliza las etiquetas y proyecta las muestras intentando maximizar la separación entre clases. Por eso, comparar ambas representaciones permite diferenciar entre la estructura natural de los datos y la separación obtenida al usar información de clase. Estas visualizaciones se complementan con métricas sencillas de separación entre grupos, de forma que el análisis no dependa únicamente de la inspección visual de las figuras.

5.5.1. Entrada y construcción del espacio de análisis

El análisis parte de los mismos archivos utilizados en el entrenamiento: `features_X.npy`, `features_y.npy`, `features_meta.csv` y `features_info.json`. Al cargarlos, el script comprueba que la matriz tenga la estructura esperada, es decir, un bloque de cinco bandas por cada canal EEG. Esta comprobación evita analizar un conjunto de características que no coincida con la salida real de la extracción.

A partir de esa matriz base se reconstruye el mismo espacio usado por los clasificadores: potencia Alpha, potencia Beta y asimetría Alpha. Para ello se aplica la función `fit_normalization_pipeline`, que calcula la referencia respecto a `NEUTRO`, ajusta el escalado y devuelve la matriz final de características. En el análisis global esta transformación se ajusta con todas las épocas disponibles, mientras que en el análisis por sujeto se ajusta de forma independiente para cada participante. Esta decisión es adecuada aquí porque el objetivo no es estimar rendimiento predictivo, sino visualizar

¹²https://en.wikipedia.org/wiki/Principal_component_analysis

¹³https://en.wikipedia.org/wiki/Linear_discriminant_analysis

y comparar la estructura interna de los datos.

5.5.2. Proyecciones globales y por sujeto

Una vez construido el espacio de análisis, el script genera las proyecciones en dos niveles. La diferencia principal entre ambos no está en la técnica utilizada, sino en el conjunto de datos sobre el que se ajusta cada representación.

- **Análisis global.** Todas las épocas se sitúan en un único espacio común. Cada proyección se dibuja dos veces: primero coloreando los puntos por condición emocional y después coloreándolos por sujeto. Esta comparación permite comprobar si las agrupaciones observadas se relacionan con JOY, NEUTRO y SAD, o si están más influidas por diferencias individuales. Las figuras generadas son `01_pca_global_condition_subject.png` y `02_lda_global_condition_subject.png`.
- **Análisis por sujeto.** Las épocas se separan por participante y, para cada uno, se calcula su propio espacio normalizado. Los resultados se muestran en cuadrículas, con un panel por sujeto y colores por condición emocional. Este análisis permite revisar si las emociones presentan estructura interna dentro de cada participante, sin que la visualización quede dominada por la variabilidad entre sujetos. Las salidas correspondientes son `03_pca_within_subject.png` y `04_lda_within_subject.png`.

5.5.3. Métricas de separabilidad

Además de las figuras, el script calcula métricas numéricas para comparar la separación de los grupos en distintos espacios. Estas métricas se calculan sobre el espacio original de características y sobre las dos proyecciones bidimensionales, de forma que se pueda contrastar si la separabilidad aparece antes o después de reducir la dimensionalidad.

La primera medida utilizada es *silhouette*¹⁴, que resume hasta qué punto cada muestra está más cerca de su propio grupo que de los demás. La segunda es el ratio entre centroides y dispersión interna, denominado en el código `centroid_ratio`. Este valor compara la distancia media entre los centros de los grupos con la dispersión media

¹⁴[https://en.wikipedia.org/wiki/Silhouette_\(clustering\)](https://en.wikipedia.org/wiki/Silhouette_(clustering))

de las muestras dentro de cada grupo; por tanto, valores más altos indican grupos más separados y compactos.

En el análisis global, estas métricas se calculan dos veces: una usando las etiquetas emocionales y otra usando los identificadores de sujeto. Esta comparación ayuda a detectar si el espacio separa mejor las emociones o a los participantes. En el análisis por sujeto, las métricas se calculan de forma independiente para cada participante utilizando únicamente las tres condiciones emocionales.

5.5.4. Resumen

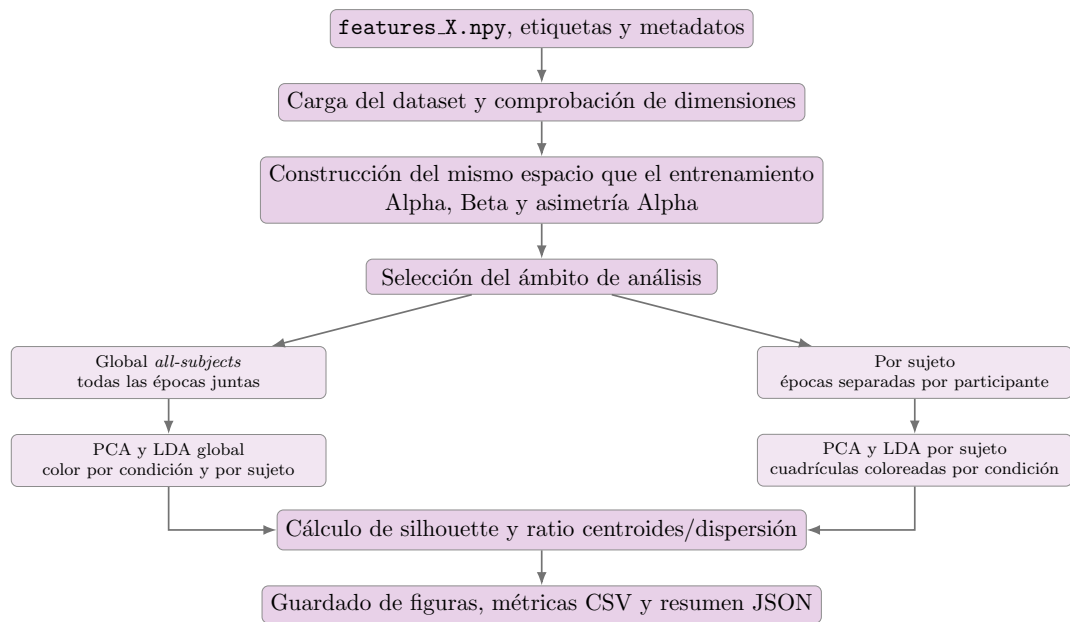


Figura 5.6: Resumen del flujo de análisis de separabilidad.

5.6. Explicabilidad de los modelos

En esta sección se describe el funcionamiento del script `explicability.py`, utilizado para analizar qué características tienen mayor influencia en las predicciones en el escenario *within-subject*. Esta fase no tiene como objetivo mejorar el rendimiento del sistema ni entrenar un nuevo modelo final, sino generar explicaciones *out-of-fold* a partir de la regresión logística entrenada en cada partición.

La explicabilidad se centró únicamente en los modelos *within-subject*, ya que el modelo general no ofrecía un comportamiento suficientemente adecuado para obtener interpretaciones fiables. La alta variabilidad entre sujetos hacía que las decisiones

del modelo global estuvieran muy influenciadas por las diferencias individuales, dificultando el análisis de patrones asociados realmente a las condiciones emocionales. En cambio, el enfoque por sujeto permite estudiar las decisiones dentro de cada participante, comparando las emociones sin mezclar esa información con la variabilidad intersujeto.

Por tanto, las explicaciones generadas en esta fase se orientan a comprender qué características Alpha, Beta y de asimetría Alpha contribuyen más a la clasificación emocional dentro de cada sujeto.

5.6.1. Cálculo de explicaciones out-of-fold

El cálculo de explicabilidad sigue la misma lógica que el entrenamiento *within-subject*: los datos se procesan por sujeto, se dividen en particiones y cada modelo se ajusta únicamente con la parte de entrenamiento. La diferencia es que aquí no se explican los modelos finales guardados en formato `joblib`, ya que estos se entrenan con todas las épocas disponibles de cada participante. En su lugar, se entrenan modelos auxiliares por fold para que cada explicación se calcule sobre muestras que no han sido vistas durante el ajuste.

Para cada sujeto se cargan las características, etiquetas y metadatos, y se construyen particiones temporales estratificadas por condición. Cada fold reserva épocas de JOY, NEUTRO y SAD, manteniendo bloques contiguos según el índice temporal de las frases. Si algún participante no dispone de suficientes épocas por clase para formar estas particiones, se omite en esta fase.

Dentro de cada fold se ajusta la normalización con las muestras de entrenamiento, se entrena una regresión logística con la misma configuración usada en el entrenamiento *within-subject* y se predicen las muestras reservadas. Sobre esas muestras de test se calculan las explicaciones SHAP, por lo que cada época explicada es *out-of-fold*.

Al tratarse de una regresión logística, el script utiliza una forma lineal de SHAP. La contribución de una característica j para una clase c se calcula como:

$$\phi_j^{(c)}(x) = w_j^{(c)} (x_j - \bar{x}_{j,\text{train}})$$

Ecuación 5.11: Contribución SHAP lineal utilizada para la regresión logística en cada fold.

donde $w_j^{(c)}$ es el coeficiente aprendido por la regresión logística para esa clase, x_j es el valor de la característica en la muestra explicada y $\bar{x}_{j,\text{train}}$ es la media de esa característica en las muestras de entrenamiento del fold. De esta forma, la explicación conserva una relación directa con los coeficientes del modelo y con los valores reales de la muestra analizada.

5.6.2. Agregación de importancias

Una vez calculadas las explicaciones de todos los folds, el script reúne los valores SHAP de cada sujeto. La agregación se realiza por clase usando las épocas cuya etiqueta real pertenece a esa condición emocional. Es decir, para resumir JOY se toman las épocas reales de JOY y las contribuciones asociadas a la salida de esa clase; el mismo criterio se aplica a NEUTRO y SAD.

Para cada combinación de sujeto, clase emocional y característica se calculan dos medidas principales. La primera es `mean_abs_shap`, que representa la magnitud media de la contribución sin tener en cuenta el signo; sirve para saber cuánto influye una característica en la decisión. La segunda es `mean_signed_shap`, que conserva el signo medio y permite distinguir si, de forma promedio, la característica empuja la predicción hacia una clase o en sentido contrario.

Después se construye un resumen global promediando las importancias entre sujetos. Este promedio no procede de un único modelo general, sino de las explicaciones calculadas dentro de cada participante. Por tanto, el resultado resume qué variables tienden a ser relevantes en el análisis *within-subject*, evitando que la interpretación quede dominada por un clasificador entrenado con todos los sujetos mezclados.

Además de guardar la importancia de cada característica individual, el código añade información descriptiva sobre su tipo. Cada variable se clasifica como potencia por banda, asimetría o característica derivada, y se asocia, cuando corresponde, a un canal, una banda de frecuencia, un par de electrodos y un cluster anatómico. Esta estructura permite generar después resúmenes más legibles que una lista completa de características.

5.6.3. Representaciones generadas

Con las tablas de importancia ya construidas, el script genera varias figuras destinadas a facilitar la revisión posterior de resultados. La primera muestra las características con mayor SHAP absoluto medio, ofreciendo una vista directa de las variables que más peso tienen en conjunto. También se calcula la importancia relativa de las bandas Alpha y Beta por clase, expresada como porcentaje dentro de las características de potencia por banda.

Para construir cada topomap, el script toma únicamente las características de potencia por canal y banda, ya que son las que pueden asociarse directamente a una posición espacial del montaje EEG. Las importancias se agregan por condición, canal y banda, y después se colocan sobre el montaje `standard_1005`. De esta forma, cada mapa representa cómo se distribuye espacialmente la contribución del modelo para una emoción y una banda concreta. Las características de asimetría no se dibujan directamente como topomap, porque pertenecen a pares de electrodos y no a un único canal. En los mapas absolutos se observa dónde se concentran las contribuciones más grandes, sin distinguir si son positivas o negativas. En los mapas firmado se conserva el signo, por lo que se puede revisar si una zona tiende a aumentar o reducir la evidencia a favor de una clase concreta.

El script también crea topomaps individuales por sujeto, útiles para comprobar si el patrón agregado es consistente o si depende mucho de algunos participantes. Por último, las importancias se agrupan por clusters cerebrales y por banda, generando un gráfico de importancia por cluster y un mapa de calor cluster-banda. La interpretación concreta de estas figuras corresponde al capítulo de resultados; aquí solo se recoge cómo se producen.

5.6.4. Guardado de resultados

Todas las salidas se almacenan en la carpeta `results/explicability/within-subject/`. El objetivo es conservar tanto las tablas detalladas como los resúmenes necesarios para consultar la explicación sin volver a ejecutar el cálculo completo.

- `shap_subject_summary.csv`: guarda, para cada sujeto explicado, el número de épocas, la exactitud, el F1 macro y la matriz de confusión *out-of-fold*.
- `shap_feature_importance_by_subject.csv`: contiene las importancias por

sujeto, clase y característica.

- `shap_feature_importance.csv`: almacena el promedio global por clase y característica, calculado a partir de los resultados individuales.
- `shap_channel_band_importance.csv`, `shap_band_importance.csv` y `shap-cluster_band_importance.csv`: recogen las agregaciones necesarias para analizar canales, bandas y clusters.
- `explicability_summary.json`: resume la configuración utilizada, las métricas medias, la matriz de confusión agregada, las características principales por clase y los porcentajes por banda y cluster.

Junto a estas tablas se guardan las figuras generadas: ranking de características, importancia relativa por banda, topomaps globales, topomaps por sujeto y visualizaciones por cluster. Estas salidas cierran el flujo de desarrollo y dejan preparada la información que se analizará en el capítulo de resultados.

5.6.5. Resumen

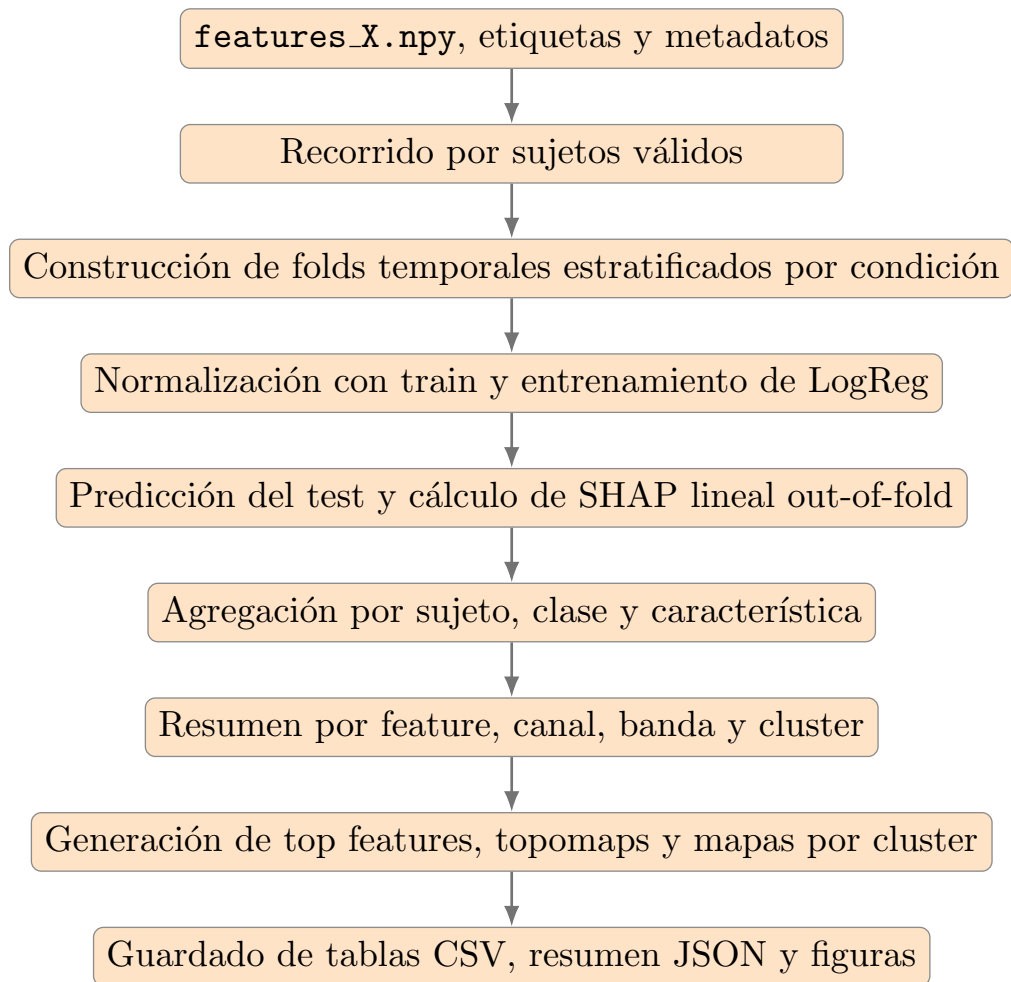


Figura 5.7: Flujo seguido por el script de explicabilidad para calcular importancias out-of-fold y generar las salidas de interpretación.

Bibliografía

- [1] Mayo Clinic. Electroencefalografía (EEG), 2024. <https://www.mayoclinic.org/es/tests-procedures/eeg/about/pac-20393875>.
- [2] Soraia M. Alarcão and Manuel J. Fonseca. Emotions recognition using eeg signals: A survey. *IEEE Transactions on Affective Computing*, 10(3):374–393, 2019. <https://ieeexplore.ieee.org/abstract/document/7946165>.
- [3] Isaac Martín de Diego, Ángel Serrano, Cristina Conde, and Enrique Cabello. Técnicas de reconocimiento automático de emociones. *Teoría de la Educación: Educación y Cultura en la Sociedad de la Información*, 7(2):92–106, 2006. <https://revistas.usal.es/tres/index.php/eks/article/view/19413/19414>.
- [4] Lin Shu, Jinyan Xie, Mingyue Yang, Ziyi Li, Zhenqi Li, Dan Liao, Xiangmin Xu, and Xinyi Yang. A review of emotion recognition using physiological signals. *Sensors*, 18(7):2074, 2018. <https://www.mdpi.com/1424-8220/18/7/2074>.
- [5] Christoph Molnar. *Interpretable Machine Learning*. 2 edition, 2022. <https://christophm.github.io/interpretable-ml-book/>.
- [6] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1:206–215, 2019. <https://www.nature.com/articles/s42256-019-0048-x>.
- [7] Psycholab. Lóbulos cerebrales para principiantes, 2021. <https://www.psycholab.com/lobulos-cerebrales-para-principiantes/>.
- [8] G. H. Klem, H. O. Lüders, H. H. Jasper, and C. Elger. The ten-twenty electrode system of the international federation. the international federation of clinical neurophysiology. *Electroencephalography and Clinical Neurophysiology. Supplement*, 52:3–6, 1999.

- [9] Robert Oostenveld and Peter Praamstra. The five percent electrode system for high-resolution eeg and erp measurements. *Clinical Neurophysiology*, 112(4):713–719, 2001. <https://www.sciencedirect.com/science/article/abs/pii/S1388245700005277>.
- [10] Carlos Novo, Leticia Chacón Gutiérrez, and José Alberto Barradas Bribiesca. Mapeo electroencefalográfico y neurofeedback, 2010. https://www.researchgate.net/figure/Figura-3-Sistema-Internacional-10-20-para-la-colocacion-de-los-electrodos_fig2_282294960.
- [11] Jennifer Delgado. Lista de 290 emociones y sentimientos: cómo diferenciarlos, 2024. <https://rinconpsicologia.com/lista-de-emociones-y-sentimientos/>.
- [12] Arjun, Aniket Singh Rajpoot, and Mahesh Raveendranatha Panicker. Subject independent emotion recognition using EEG signals employing attention driven neural networks. *Biomedical Signal Processing and Control*, 75:103547, 2022. <https://www.sciencedirect.com/science/article/abs/pii/S1746809422000696>.
- [13] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006.
- [14] Fabien Lotte, Marco Congedo, Anatole Lécuyer, Fabrice Lamarche, and Bruno Arnaldi. A review of classification algorithms for EEG-based brain-computer interfaces. *Journal of Neural Engineering*, 4(2):R1–R13, 2007. <https://iopscience.iop.org/article/10.1088/1741-2560/4/2/R01>.
- [15] Ignacio Moreno Hojas. Regresión logística en python, s. f. <https://www.statdeveloper.com/regresion-logistica-en-python/>.
- [16] Codificando Bits. Qué es la regresión multiclase, s. f. <https://codificandobits.com/blog/regresion-multiclase/>.
- [17] adrigm. Crear un adivinador, 2013. <https://www.genbeta.com/desarrollo/crear-un-adivinador>.
- [18] Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, pages 1137–1145. Morgan Kaufmann, 1995. <https://www.ijcai.org/Proceedings/95-2/Papers/016.pdf>.

- [19] Gladys Andrea Rodríguez Guerrero. Cross-validation, 2021. <https://gladysandrea-rodriguez.medium.com/cross-validation-11e9ea688506>.
- [20] Geoffrey Brookshire, Jake Kasper, Nicholas M. Blauch, Yunan Charles Wu, Ryan Glatt, David A. Merrill, Spencer Gerrol, Keith J. Yoder, Colin Quirk, and Ché Lucero. Data leakage in deep learning studies of translational EEG. *Frontiers in Neuroscience*, 18:1373515, 2024. <https://doi.org/10.3389/fnins.2024.1373515>.
- [21] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):15, 2012. <https://doi.org/10.1145/2382577.2382579>.
- [22] scikit-learn developers. Metrics and scoring: Quantifying the quality of predictions, 2026. https://scikit-learn.org/stable/modules/model_evaluation.html.
- [23] Ben Hayes. Demystifying the confusion matrix, 2018. <https://benhay.es/posts/demystifying-confusion-matrix/>.
- [24] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017. <https://proceedings.neurips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf>.
- [25] Alexandre Gramfort, Martin Luessi, Eric Larson, Denis A. Engemann, Daniel Strohmeier, Christian Brodbeck, Roman Goj, Mainak Jas, Teon Brooks, Lauri Parkkonen, and Matti S. Hämäläinen. MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, 7:267, 2013. <https://www.frontiersin.org/articles/10.3389/fnins.2013.00267/full>.
- [26] Jose Antonio Urigüen and Begonya Garcia-Zapirain. EEG artifact removal—state-of-the-art and guidelines. *Journal of Neural Engineering*, 12(3):031001, 2015. <https://doi.org/10.1088/1741-2560/12/3/031001>.
- [27] Marta Aguilar Díaz. Desarrollo de aplicación automática de señales EEG. Trabajo fin de grado en ingeniería electrónica industrial y automática, Escuela Superior de Ciencias Experimentales y Tecnología, 2026. Tutor: Álvaro García López. Curso académico 2025/26.

- [28] EEGLAB. Time-frequency analysis tutorial, 2026. https://eeglab.org/tutorials/ConceptsGuide/Time_frequency_tutorial.html.
- [29] Peter D. Welch. The use of fast fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, 1967. <https://doi.org/10.1109/TAU.1967.1161901>.
- [30] Sander Koelstra, Christian Mühl, Mohammad Soleymani, Jong-Seok Lee, Ashkan Yazdani, Touradj Ebrahimi, Thierry Pun, Anton Nijholt, and Ioannis Patras. DEAP: A database for emotion analysis using physiological signals. *IEEE Transactions on Affective Computing*, 3(1):18–31, 2012. <https://doi.org/10.1109/T-AFFC.2011.15>.
- [31] Bujar Raufi and Luca Longo. An evaluation of the EEG alpha-to-theta and theta-to-alpha band ratios as indexes of mental workload. *Frontiers in Neuroinformatics*, 16:861967, 2022. <https://doi.org/10.3389/fninf.2022.861967>.
- [32] Ezra E. Smith, Samantha J. Reznik, Jennifer L. Stewart, and John J. B. Allen. Assessing and conceptualizing frontal EEG asymmetry: An updated primer on recording, processing, analyzing, and interpreting frontal alpha asymmetry. *International Journal of Psychophysiology*, 111:98–114, 2017. <https://doi.org/10.1016/j.ijpsycho.2016.11.005>.
- [33] Ante Topic and Mladen Russo. Emotion recognition based on EEG feature maps through deep learning network. *Engineering Science and Technology, an International Journal*, 24(6):1442–1454, 2021. <https://www.sciencedirect.com/science/article/pii/S2215098621000768>.
- [34] MNE-Python developers. MNE-Python Documentation, 2026. <https://mne.tools/stable/index.html>.
- [35] NumPy developers. NumPy Documentation, 2026. <https://numpy.org/doc/stable/>.
- [36] pandas developers. pandas Documentation, 2026. <https://pandas.pydata.org/docs/>.
- [37] SciPy developers. SciPy Documentation, 2026. <https://docs.scipy.org/doc/scipy/>.

- [38] scikit-learn developers. scikit-learn Documentation, 2026. <https://scikit-learn.org/stable/>.
- [39] Matplotlib developers. Matplotlib Documentation, 2026. <https://matplotlib.org/stable/index.html>.
- [40] joblib developers. joblib Documentation, 2026. <https://joblib.readthedocs.io/en/stable/>.